

Dense and CSR Sparse matrix library in C++ leveraging Intel oneMKL (CBLAS)

Generated by Doxygen 1.9.8

Chapter 1

Dense and CSR Sparse matrix library in C++ leveraging Intel oneMKL (CBLAS)

1.1 What this project does

In this project of ours we are developing a high-performance Dense and CSR Sparse matrix library in modern C++ (icpx), leveraging Intel oneMKL (CBLAS + MKL Sparse BLAS) for numerical computing and HPC applications.

1.2 How to build & compile the project

The following build and compile instructions are for Linux (we will assume that you are using Ubuntu/Debian like distros). If you use Windows you may encounter problems.

1.2.1 Requirements

Make sure you have **all of the following installed** before reading the Setup Instructions section.

- **Git:**

<https://git-scm.com/install/linux>

or

```
sudo apt-get install git
```

- **CMake:**

```
sudo apt-get install cmake
```

- **Intel oneAPI Base Toolkit:**

<https://www.intel.com/content/www/us/en/developer/tools/oneapi/base-toolkit-download.html>

- **VS Code** (or you IDE of preference but we will assume you use VS Code below)

You usually can install it directly from the App Center or from [here](#)

1.2.2 Setup Instructions

1. First you will need to clone the repository to your desired destination.

Go to the directory you want to clone the project:

```
cd <path_to_your_dir>
```

Clone the repo:

```
git clone https://github.com/VictorMeimariss/Cpp-Project.git
```

Then get inside the project root folder: `cd Cpp-Project`

2. Next, you need to source/setup the oneAPI variables:

```
source /opt/intel/oneapi/setvars.sh
```

The path above is usually the default, it may vary if you changed anything in the installation process

3. Open your IDE:

For VS Code do:

```
code .
```

1.2.3 Build & Linking Instructions

Now to run the CMake Configuration run the following command in the VScode terminal:

```
cmake -S . -B obj
```

this will create a directory called `/obj` where all the `.o` files will go.

Next, Build and Link everything with the following command:

```
cmake --build obj
```

1.2.4 Run a test (optional)

Now to confirm that everything went well you can run one of the test functions `.cpp` files (all the executables are inside `/bin`)

```
./bin/test_CG_dsolve
```

it doesn't need to be `test_CG_dsolve`, this was just used as an example.

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

dense::matrix< T >	A class to create a dense matrix object of any number of dimensions	??
sparse::matrix< T >	A class to create a sparse matrix object in csr form	??

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

/home/johnzik/Dev/Cpp-Project/inc/ cross.hpp	??
Header file for the cross product functions of dense and sparse matrices	??
/home/johnzik/Dev/Cpp-Project/inc/ def.hpp	??
Wrapper header file for the blas functions	??
/home/johnzik/Dev/Cpp-Project/inc/ dense.hpp	??
Header file for the matrix class for dense matrices and its associated functions	??
/home/johnzik/Dev/Cpp-Project/inc/ matrix.hpp	??
Header that connects all header files	??
/home/johnzik/Dev/Cpp-Project/inc/ mmio.h	??
/home/johnzik/Dev/Cpp-Project/inc/ sparse.hpp	??
Header file for the matrix class for sparse matrices and its associated functions	??
/home/johnzik/Dev/Cpp-Project/src/ axpy_add.cpp	??
/home/johnzik/Dev/Cpp-Project/src/ axpy_sub.cpp	??
/home/johnzik/Dev/Cpp-Project/src/ CG_dsolve.cpp	??
/home/johnzik/Dev/Cpp-Project/src/ copy_constructor.cpp	??
/home/johnzik/Dev/Cpp-Project/src/ dense_AssignmentOP.cpp	??
/home/johnzik/Dev/Cpp-Project/src/ dense_diagonal_construct.cpp	??
/home/johnzik/Dev/Cpp-Project/src/ dense_readMM.cpp	??
/home/johnzik/Dev/Cpp-Project/src/ dense_vm_gemv.cpp	??
/home/johnzik/Dev/Cpp-Project/src/ exp_operator.cpp	??
/home/johnzik/Dev/Cpp-Project/src/ FGMRES_dsolve.cpp	??
/home/johnzik/Dev/Cpp-Project/src/ gemm_high.cpp	??
/home/johnzik/Dev/Cpp-Project/src/ gemm_low.cpp	??
/home/johnzik/Dev/Cpp-Project/src/ gemv.cpp	??
/home/johnzik/Dev/Cpp-Project/src/ get_dim_size.cpp	??
/home/johnzik/Dev/Cpp-Project/src/ get_nod.cpp	??
/home/johnzik/Dev/Cpp-Project/src/ get_size.cpp	??
/home/johnzik/Dev/Cpp-Project/src/ get_value_at.cpp	??
/home/johnzik/Dev/Cpp-Project/src/ get_values_data.cpp	??
/home/johnzik/Dev/Cpp-Project/src/ get_values_data_const.cpp	??
/home/johnzik/Dev/Cpp-Project/src/ identity_constructor.cpp	??
/home/johnzik/Dev/Cpp-Project/src/ low_level_multiplication.cpp	??
/home/johnzik/Dev/Cpp-Project/src/ LU_solver.cpp	??
/home/johnzik/Dev/Cpp-Project/src/ mmio.c	??
/home/johnzik/Dev/Cpp-Project/src/ print2D.cpp	??

/home/johnzik/Dev/Cpp-Project/src/print_matrix.cpp	??
/home/johnzik/Dev/Cpp-Project/src/scal_mn.cpp	??
/home/johnzik/Dev/Cpp-Project/src/scal_nm.cpp	??
/home/johnzik/Dev/Cpp-Project/src/set_value_at.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse-diagonal-getter.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse-get-value.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse-parentheses-operator.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse-set-value.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse_add.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse_assignment_operator.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse_banded_matrix.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse_comparison_operator.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse_constant_mul.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse_copy_constructor.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse_handle_constructor.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse_identity_matrix.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse_mm_dxs.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse_mm_sxd.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse_mul_nxs.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse_mul_sxn.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse_mv.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse_print2D.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse_readMM.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse_refresh_handle.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse_shape.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse_spmm.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse_sub.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse_transpose.cpp	??
/home/johnzik/Dev/Cpp-Project/src/sparse_vxs.cpp	??
/home/johnzik/Dev/Cpp-Project/src/svd.cpp	??
/home/johnzik/Dev/Cpp-Project/src/transpose.cpp	??
/home/johnzik/Dev/Cpp-Project/src/vectors_constructor.cpp	??

Chapter 4

Class Documentation

4.1 `dense::matrix< T >` Class Template Reference

A class to create a dense matrix object of any number of dimensions.

```
#include <dense.hpp>
```

Public Member Functions

- `matrix ()`
Default constructor. Initializes an empty matrix with zero dimensions and size.
- `matrix (std::vector< T > diag, int n, int m)`
Constructs a diagonal matrix.
- `template<typename... Dims>`
`matrix (Dims... dims)`
Constructor with variadic template input for dimensions.
- `matrix (matrix &&A) noexcept`
Move constructor, transfers the ownership of matrix A.
- `matrix (matrix const &A)`
Copy constructor, copies matrix A.
- `~matrix ()=default`
Destructor, deletes data associated with the matrix.
- `matrix (std::vector< T > v, std::vector< size_t > d, std::vector< int > p)`
Vectors constructor, builds a new matrix from the 3 vectors.
- `matrix I (int N)`
Constructs an N by N identity matrix.
- `matrix & operator= (matrix const &A)`
Operator =, creates a deep copy of the matrix and assigns it.
- `matrix operator+ (matrix const &A)`
Operator +, adds the values of two matrices of same size and dimensions.
- `template<typename U = T>`
`std::enable_if_t< std::is_same_v< U, T >, matrix< T > > operator* (const matrix< T > &A) const`
*Operator *, multiplies two 2D dense matrices.*
- `bool operator== (matrix const &A)`
Operator ==, compares two matrices.

- `matrix operator* (T x)`
Operator *, multiplies a matrix by a number.
- `std::vector< T > operator* (const std::vector< T > x)`
Operator *, multiplies a matrix of n columns by a vector of n rows.
- `matrix operator- (matrix const &A)`
Operator -, subtracts the values of second matrix from the first, both must have same size and dimensions.
- `matrix transpose () const`
Function that transposes the matrix in 2D.
- `matrix operator^ (int n)`
Returns A^n .
- `matrix operator/ (matrix const &A)`
Operator /, Performs column-wise augmentation.
- `matrix operator| (matrix const &A)`
Operator |, Performs row-wise augmentation.
- `T & operator() (size_t index)`
Operator (), returns a writable reference to the value at the coordinates. Variadic index.
- `template<typename... Indices>`
`const T & operator() (Indices... indices)`
Operator (), returns a constant (non-writable) reference to the value at the coordinates. Variadic index.
- `size_t get_size () const`
Gets size of matrix.
- `size_t get_nod () const`
Gets number of dimensions.
- `size_t get_dim_size (size_t index) const`
Gets specified dimension's size using an index.
- `T get_value_at (size_t index) const`
Gets value at a given index.
- `void svd (matrix< T > &u, matrix< T > &s, matrix< T > &v)`
Computes the SVD of the matrix.
- `T * get_values_data ()`
Gets all values data.
- `void set_value_at (size_t index, T value)`
Sets value at a given index.
- `template<typename... Types>`
`void set_values (Types &&... values)`
Sets all matrix values, clearing any previously stored values and then filling with zeros for the rest.
- `const T * get_const_values_data () const`
returns a constant pointer to the underlying vector array values
- `void print2D ()`
Prints the values of 2D array.
- `template<typename... Dims>`
`void reshape (Dims... dims)`
Reshapes matrix after default constructor initialisation.
- `void low_level_matrix_multiplication (const CBLAS_TRANSPOSE transa, const CBLAS_TRANSPOSE transb, const T beta, matrix< T > const &A, matrix< T > const &B, matrix< T > &C, const T alpha)`
Low level implementation of matrix multiplication using BLAS. documentation at <https://www.intel.com/content/www/us/en/docs/onemkl/developer-reference-c/2024-1/cblas-gemm-001.html#GUID-97718E5C-6E0A-44F0-B2B1-A551F0F164B2>.

Static Public Member Functions

- [static matrix dense_readMM](#) ([const](#) std::string &[MMpath](#))

Reads a Matrix Market (MM) sparse matrix, converts it from COO to CSR format and returns a CSR matrix obj (with a handle constructor)

4.1.1 Detailed Description

```
template<typename T>
class dense::matrix< T >
```

A class to create a dense matrix object of any number of dimensions.

Template Parameters

<i>T</i>	The scalar type used for matrix elements.
----------	---

Definition at line 32 of file [dense.hpp](#).

4.1.2 Constructor & Destructor Documentation

4.1.2.1 matrix() [1/6]

```
template<typename T >
dense::matrix< T >::matrix ( ) [inline]
```

Default constructor. Initializes an empty matrix with zero dimensions and size.

Definition at line 54 of file [dense.hpp](#).

4.1.2.2 matrix() [2/6]

```
template<typename T >
matrix::matrix (
    std::vector< T > diag,
    int n,
    int m )
```

Constructs a diagonal matrix.

Parameters

<i>diag</i>	An std vector storing the diagonal elements
<i>n</i>	The number of rows
<i>m</i>	The number of columns

Definition at line 5 of file [dense_diagonal_construct.cpp](#).

4.1.2.3 matrix() [3/6]

```
template<typename T >
template<typename... Dims>
dense::matrix< T >::matrix (
    Dims... dims ) [inline]
```

Constructor with variadic template input for dimensions.

Template Parameters

<i>Dims</i>	A variadic pack representing dimension sizes.
-------------	---

Parameters

<i>dims</i>	The sizes of each dimension.
-------------	------------------------------

Definition at line 69 of file [dense.hpp](#).

4.1.2.4 matrix() [4/6]

```
template<typename T >
dense::matrix< T >::matrix (
    matrix< T > && A ) [inline], [noexcept]
```

Move constructor, transfers the ownership of matrix A.

Parameters

<i>A</i>	A matrix reference to be moved
----------	--------------------------------

Definition at line 92 of file [dense.hpp](#).

4.1.2.5 matrix() [5/6]

```
template<typename T >
matrix::matrix (
    matrix< T > const & A )
```

Copy constructor, copies matrix A.

Parameters

<i>A</i>	A constant matrix reference to be copied
----------	--

Definition at line 5 of file [copy_constructor.cpp](#).

4.1.2.6 matrix() [6/6]

```
template<typename T >
matrix::matrix (
    std::vector< T > v,
    std::vector< size_t > d,
    std::vector< int > p )
```

Vectors constructor, builds a new matrix from the 3 vectors.

Parameters

<i>v</i>	vector storing all matrix values.
<i>d</i>	storing the size of each dimension.
<i>p</i>	storing the number of elements separating adjacent values in each dimension of the tensor

Definition at line 5 of file [vectors_constructor.cpp](#).

4.1.3 Member Function Documentation

4.1.3.1 dense_readMM()

```
template<typename T >
matrix< T > matrix::dense_readMM (
    const std::string & MMpath ) [static]
```

Reads a Matrix Market (MM) sparse matrix, converts it from COO to CSR format and returns a CSR matrix obj (with a handle cosnstructor)

Parameters

<i>MMpath</i>	A string with the path to the MM matrix
---------------	---

Returns

A sparse matrix object created with the handle constructor

Definition at line 6 of file [dense_readMM.cpp](#).

4.1.3.2 get_const_values_data()

```
template<typename T >
template const std::complex< double > * dense::matrix< T >::get_const_values_data ( ) const
```

returns a constant pointer to the underlying vector array values

Definition at line 5 of file [get_values_data_const.cpp](#).

4.1.3.3 get_dim_size()

```
template<typename T >
template size_t dense::matrix< T >::get_dim_size (
    size_t index ) const
```

Gets specified dimension's size using an index.

Parameters

<i>index</i>	The linear 0-based index in the dim_size vector.
--------------	--

Definition at line 5 of file [get_dim_size.cpp](#).

4.1.3.4 get_nod()

```
template<typename T >
template size_t dense::matrix< T >::get_nod ( ) const
```

Gets number of dimensions.

Definition at line 8 of file [get_nod.cpp](#).

4.1.3.5 get_size()

```
template<typename T >
size_t matrix::get_size ( ) const
```

Gets size of matrix.

Definition at line 5 of file [get_size.cpp](#).

4.1.3.6 get_value_at()

```
template<typename T >
T matrix::get_value_at (
    size_t index ) const
```

Gets value at a given index.

Parameters

<i>index</i>	The linear 0-based index in the flat values vector.
--------------	---

Definition at line 5 of file [get_value_at.cpp](#).

4.1.3.7 get_values_data()

```
template<typename T >
```

```
T * matrix::get_values_data ( )
```

Gets all values data.

Definition at line 5 of file [get_values_data.cpp](#).

4.1.3.8 I()

```
template<typename T >
template dense::matrix< std::complex< double > > matrix::I (
    int N )
```

Constructs an N by N identity matrix.

Parameters

<i>N</i>	the number of rows/columns in the square identity matrix
----------	--

Definition at line 5 of file [identity_constructor.cpp](#).

4.1.3.9 low_level_matrix_multiplication()

```
template<typename T >
template void dense::matrix< T >::low_level_matrix_multiplication (
    const CBLAS_TRANSPOSE transa,
    const CBLAS_TRANSPOSE transb,
    const T beta,
    matrix< T > const & A,
    matrix< T > const & B,
    matrix< T > & C,
    const T alpha )
```

Low level implementation of matrix multiplication using BLAS. documentation at <https://www.intel.com/content/www/us/en/docs/onemkl/developer-reference-c/2024-1/cblas-gemm-001.html#GUID-97718E5C-6E0A-44F0-B2B1-A551F0F164B2>.

Template Parameters

<i>T</i>	encompasses matrix type
----------	-------------------------

Definition at line 5 of file [low_level_multiplication.cpp](#).

4.1.3.10 operator()() [1/2]

```
template<typename T >
template<typename... Indices>
const T & dense::matrix< T >::operator() (
    Indices... indices )
```

Operator (), returns a constant (non-writable) reference to the value at the coordinates. Variadic index.

Template Parameters

<i>index</i>	variadic argument for the coordinates of the value to be accessed.
--------------	--

4.1.3.11 operator>() [2/2]

```
template<typename T >
T & dense::matrix< T >::operator() (
    size_t index )
```

Operator (), returns a writable reference to the value at the coordinates. Variadic index.

Template Parameters

<i>index</i>	variadic argument for the coordinates of the value to be accessed
--------------	---

4.1.3.12 operator*() [1/3]

```
template<typename T >
template<typename U >
std::enable_if_t< std::is_same_v< U, T >, matrix< T > > matrix::operator* (
    const matrix< T > & A ) const
```

Operator *, multiplies two 2D dense matrices.

Parameters

<i>A</i>	A constant matrix reference used after the operator
----------	---

Definition at line 7 of file [gemm_high.cpp](#).

4.1.3.13 operator*() [2/3]

```
template<typename T >
std::vector< T > matrix::operator* (
    const std::vector< T > x )
```

Operator *, multiplies a matrix of n columns by a vector of n rows.

Parameters

<i>x</i>	A vector used after the operator
----------	----------------------------------

Template Parameters

<i>T</i>	encompasses all primitive numerical data types //Vector
----------	---

Definition at line 5 of file [gemv.cpp](#).

4.1.3.14 operator*() [3/3]

```
template<typename T >
matrix< T > matrix::operator* (
    T x )
```

Operator *, multiplies a matrix by a number.

Parameters

<i>x</i>	A numerical primitive used after the operator
----------	---

Template Parameters

<i>T</i>	encompasses all primitive numerical data types //Vector
----------	---

Definition at line 5 of file [scal_mn.cpp](#).

4.1.3.15 operator+()

```
template<typename T >
matrix< T > matrix::operator+ (
    matrix< T > const & A )
```

Operator +, adds the values of two matrices of same size and dimensions.

Parameters

<i>A</i>	A constant matrix reference used after the operator
----------	---

Definition at line 5 of file [axpy_add.cpp](#).

4.1.3.16 operator-()

```
template<typename T >
matrix< T > matrix::operator- (
    matrix< T > const & A )
```

Operator -, subtracts the values of second matrix from the first, both must have same size and dimensions.

Parameters

<i>A</i>	A constant matrix reference used after the operator
----------	---

Definition at line 5 of file [axpy_sub.cpp](#).

4.1.3.17 operator/()

```
template<typename T >
matrix< T >::operator/ (
    matrix< T > const & A )
```

Operator /, Performs column-wise augmentation.

Parameters

A	A constant matrix reference to be combined.
----------	---

4.1.3.18 operator=()

```
template<typename T >
matrix< T > & matrix::operator= (
    matrix< T > const & A )
```

Operator =, creates a deep copy of the matrix and assigns it.

Parameters

A	A constant matrix reference used after the operator
----------	---

Definition at line 5 of file [dense_AssignmentOP.cpp](#).

4.1.3.19 operator==()

```
template<typename T >
bool dense::matrix< T >::operator== (
    matrix< T > const & A ) [inline]
```

Operator ==, compares two matrices.

Parameters

A	A constant matrix reference used after the operator
----------	---

Definition at line 144 of file [dense.hpp](#).

4.1.3.20 operator^()

```
template<typename T >
matrix< T > matrix::operator^ (
    int n )
```

Returns A^n .

Parameters

<i>A</i>	n integer power, negative values return $(\text{inv}(A))^{(-n)}$
----------	--

Definition at line 6 of file [exp_operator.cpp](#).

4.1.3.21 operator" | ()

```
template<typename T >
matrix dense::matrix< T >::operator| (
    matrix< T > const & A )
```

Operator |, Performs row-wise augmentation.

Parameters

<i>A</i>	A constant matrix reference to be combined.
----------	---

4.1.3.22 print2D()

```
template<typename T >
template void dense::matrix< T >::print2D ( )
```

Prints the values of 2D array.

Definition at line 6 of file [print2D.cpp](#).

4.1.3.23 reshape()

```
template<typename T >
template<typename... Dims>
void dense::matrix< T >::reshape (
    Dims... dims ) [inline]
```

Reshapes matrix after default constructor initialisation.

Template Parameters

<i>Dims</i>	A variadic pack representing dimension sizes.
-------------	---

Parameters

<i>dims</i>	The sizes of each dimension.
-------------	------------------------------

Definition at line 285 of file [dense.hpp](#).

4.1.3.24 set_value_at()

```
template<typename T >
void dense::matrix< T >::set_value_at (
    size_t index,
    T value )
```

Sets value at a given index.

Parameters

<i>index</i>	The linear 0-based index in the flat values vector.
<i>value</i>	The templated value to set.
<i>index</i>	The linear 0-based index in the flat values vector.
<i>value</i>	The long double value to set.

Definition at line 10 of file [set_value_at.cpp](#).

4.1.3.25 set_values()

```
template<typename T >
template<typename... Types>
void dense::matrix< T >::set_values (
    Types &&... values ) [inline]
```

Sets all matrix values, clearing any previously stored values and then filling with zeros for the rest.

Template Parameters

<i>Types</i>	Variadic template arguments for the values to set.
--------------	--

Parameters

<i>values</i>	The values to set into the matrix
---------------	-----------------------------------

Definition at line 263 of file [dense.hpp](#).

4.1.3.26 svd()

```
template<typename T >
void matrix::svd (
    matrix< T > & u,
    matrix< T > & s,
    matrix< T > & v )
```

Computes the SVD of the matrix.

Parameters

<i>u</i>	A dense matrix, will store the left side singular vectors
<i>s</i>	A dense matrix, will store the singular values in its main diagonal
<i>v</i>	A dense matrix, will store the right side singular vectors

Definition at line 7 of file [svd.cpp](#).

4.1.3.27 `transpose()`

```
template<typename T >
matrix< T > matrix::transpose ( ) const
```

Function that transposes the matrix in 2D.

Definition at line 5 of file [transpose.cpp](#).

The documentation for this class was generated from the following files:

- [/home/johnzik/Dev/Cpp-Project/inc/dense.hpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/axpy_add.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/axpy_sub.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/copy_constructor.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/dense_AssignmentOP.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/dense_diagonal_construct.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/dense_readMM.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/exp_operator.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/gemm_high.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/gemv.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/get_dim_size.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/get_nod.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/get_size.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/get_value_at.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/get_values_data.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/get_values_data_const.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/identity_constructor.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/low_level_multiplication.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/print2D.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/scal_mn.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/set_value_at.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/svd.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/transpose.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/vectors_constructor.cpp](#)

4.2 `sparse::matrix< T >` Class Template Reference

A class to create a sparse matrix object in csr form.

```
#include <sparse.hpp>
```

Public Member Functions

- **matrix ()=default**
Default sparse matrix constructor.
- **void refresh_handle () const**
rebuilds a valid handle according to csr vectors.
- **template<typename U >**
matrix (const std::vector< T > &values, const std::vector< U > &col_idx, const std::vector< U > &row_idx, size_t rows, size_t cols)
Sparse matrix constructor.
- **matrix (const sparse_matrix_t h, size_t rows, size_t cols, bool own)**
Sparse matrix object constructor from MKL handle.
- **~matrix ()**
Sparse matrix destructor.
- **matrix (matrix &&A) noexcept**
Move constructor, transfers the ownership of matrix A.
- **matrix (matrix const &A)**
Matrix copy constructor.
- **matrix & operator= (matrix const &A)**
Operator =, creates a deep copy of the matrix and assigns it.
- **matrix operator+ (matrix const &A)**
Operator +, adds the values of two matrices of same size and dimensions.
- **matrix operator* (matrix const &A)**
*Operator *, multiplies two 2D sparse matrices.*
- **matrix operator* (T x)**
*Operator *, multiplies a sparse matrix by a number.*
- **std::vector< T > operator* (const std::vector< T > x)**
*Operator *, multiplies a sparse matrix by a vector.*
- **matrix operator- (matrix const &A)**
Operator -, subtracts the values of second matrix from the first, both must have same size and dimensions.
- **bool operator== (matrix const &A)**
Operator ==, compares two matrices.
- **T get_value_at (size_t index) const**
Gets value at a given index.
- **void set_value_at (size_t index, T value)**
Sets value at a given index.
- **const T operator() (size_t r, size_t c) const**
Returns a constant value in the matrix.
- **std::pair< size_t, size_t > shape () const**
Returns the shape (dimensions) of a matrix.
- **std::vector< T > diag () const**
Returns a constant vector reference containing the diagonal elements of a matrix. For reading.
- **void print2D ()**
Prints the values of 2D array.
- **void print_matrix ()**
Prints the sparse matrix row by row.
- **const sparse_matrix_t & get_handle () const**
Returns MKL handle (const access for read-only use in operations).
- **void invalidate_handle ()**
Invalidates handle.
- **const std::vector< T > get_values_data_const () const**

Gets values' data const.

- `std::vector< T > & get_values_data ()`

Gets values' non const.

- `matrix transpose () const`

Function that transposes the matrix in 2D.

Static Public Member Functions

- `static matrix I (int N)`

Constructs an N by N identity matrix.

- `static matrix banded (int N, int kl, int ku, T val)`

Constructs an N by N banded matrix.

- `static matrix sparse_readMM (const std::string &MMpath)`

Reads a Matrix Market (MM) sparse matrix, converts it from COO to CSR format and returns a CSR matrix obj (with a handle constructor)

4.2.1 Detailed Description

```
template<typename T>
class sparse::matrix< T >
```

A class to create a sparse matrix object in csr form.

Template Parameters

<code>T</code>	The scalar type used for matrix elements.
----------------	---

Definition at line 33 of file `sparse.hpp`.

4.2.2 Constructor & Destructor Documentation

4.2.2.1 `matrix()` [1/4]

```
template<typename T >
template<typename U >
sparse::matrix< T >::matrix (
    const std::vector< T > & values,
    const std::vector< U > & col_idx,
    const std::vector< U > & row_idx,
    size_t rows,
    size_t cols ) [inline]
```

Sparse matrix constructor.

Parameters

<code>values</code>	sets the values vector
<code>col_idx</code>	sets the col_idx vector
<code>row_idx</code>	sets the row_idx vector
<code>rows</code>	sets row size
<code>cols</code>	sets column size

Definition at line 95 of file [sparse.hpp](#).

4.2.2.2 matrix() [2/4]

```
template<typename T >
matrix::matrix (
    const sparse_matrix_t h,
    size_t rows,
    size_t cols,
    bool own )
```

Sparse matrix object constructor from MKL handle.

Parameters

<i>h</i>	MKL sparse matrix handle.
<i>rows</i>	number of rows.
<i>cols</i>	number of columns.
<i>own</i>	CSR arrays ownership.

Definition at line 6 of file [sparse_handle_constructor.cpp](#).

4.2.2.3 ~matrix()

```
template<typename T >
sparse::matrix< T >::~~matrix ( ) [inline]
```

Sparse matrix destructor.

Definition at line 111 of file [sparse.hpp](#).

4.2.2.4 matrix() [3/4]

```
template<typename T >
sparse::matrix< T >::matrix (
    matrix< T > && A ) [inline], [noexcept]
```

Move constructor, transfers the ownership of matrix A.

Parameters

<i>A</i>	A matrix reference to be moved
----------	--------------------------------

Definition at line 121 of file [sparse.hpp](#).

4.2.2.5 matrix() [4/4]

```
template<typename T >
```



```
matrix::matrix (
    matrix< T > const & A )
```

Matrix copy constructor.

Parameters

<i>A</i>	A constant sparse matrix reference to be copied
----------	---

Definition at line 5 of file [sparse_copy_constructor.cpp](#).

4.2.3 Member Function Documentation

4.2.3.1 `banded()`

```
template<typename T >
matrix< T > matrix::banded (
    int N,
    int kl,
    int ku,
    T val ) [static]
```

Contructs an N by N banded matrix.

Parameters

<i>N</i>	The number of rows/columns
<i>kl</i>	The number of elements below the main diagonal
<i>ku</i>	The number of the elements above the main diagonal
<i>val</i>	The type <T> value of all the bands

Definition at line 5 of file [sparse_banded_matrix.cpp](#).

4.2.3.2 `diag()`

```
template<typename T >
std::vector< T > matrix::diag ( ) const
```

Returns a constant vector reference containing the diagonal elements of a matrix. For reading.

Definition at line 5 of file [sparse-diagonal-getter.cpp](#).

4.2.3.3 `get_handle()`

```
template<typename T >
const sparse_matrix_t & sparse::matrix< T >::get_handle ( ) const [inline]
```

Returns MKL handle (const access for read-only use in operations).

Definition at line 242 of file [sparse.hpp](#).

4.2.3.4 get_value_at()

```
template<typename T >
T matrix::get_value_at (
    size_t index ) const
```

Gets value at a given index.

Parameters

<i>index</i>	The linear 0-based index in the flat values vector.
--------------	---

Definition at line 5 of file [sparse-get-value.cpp](#).

4.2.3.5 get_values_data()

```
template<typename T >
std::vector< T > & sparse::matrix< T >::get_values_data ( ) [inline]
```

Gets values' non const.

Definition at line 257 of file [sparse.hpp](#).

4.2.3.6 get_values_data_const()

```
template<typename T >
const std::vector< T > sparse::matrix< T >::get_values_data_const ( ) const [inline]
```

Gets values' data const.

Definition at line 252 of file [sparse.hpp](#).

4.2.3.7 I()

```
template<typename T >
matrix< T > matrix::I (
    int N ) [static]
```

Constructs an N by N identity matrix.

Parameters

<i>N</i>	the number of rows/columns in the square identity matrix
----------	--

Definition at line 5 of file [sparse_identity_matrix.cpp](#).

4.2.3.8 invalidate_handle()

```
template<typename T >
```

```
void sparse::matrix< T >::invalidate_handle ( ) [inline]
```

Invalidates handle.

Definition at line 247 of file [sparse.hpp](#).

4.2.3.9 `operator()()`

```
template<typename T >
const T matrix::operator() (
    size_t r,
    size_t c ) const
```

Returns a constant value in the matrix.

Parameters

<i>c</i>	The column of the value to be returned.
<i>r</i>	The row of the value to be returned.

Definition at line 5 of file [sparse-parentheses-operator.cpp](#).

4.2.3.10 `operator*()` [1/3]

```
template<typename T >
std::vector< T > matrix::operator* (
    const std::vector< T > x )
```

Operator *, multiplies a sparse matrix by a vector.

Parameters

<i>x</i>	A vector used after the operator
----------	----------------------------------

Template Parameters

<i>T</i>	encompasses all primitive numerical data types
----------	--

Definition at line 5 of file [sparse_mv.cpp](#).

4.2.3.11 `operator*()` [2/3]

```
template<typename T >
matrix< T > matrix::operator* (
    matrix< T > const & A )
```

Operator *, multiplies two 2D sparse matrices.

Parameters

A	A constant matrix reference used after the operator
-----	---

Definition at line 5 of file [sparse_spmv.cpp](#).

4.2.3.12 operator*() [3/3]

```
template<typename T >
matrix< T > matrix::operator* (
    T x )
```

Operator *, multiplies a sparse matrix by a number.

Parameters

x	A numerical primitive used after the operator
-----	---

Template Parameters

T	encompasses all primitive numerical data types
-----	--

Definition at line 5 of file [sparse_constant_mul.cpp](#).

4.2.3.13 operator+()

```
template<typename T >
sparse::matrix< T > sparse::matrix< T >::operator+ (
    matrix< T > const & A )
```

Operator +, adds the values of two matrices of same size and dimensions.

Parameters

A	A constant matrix reference used after the operator
-----	---

Definition at line 5 of file [sparse_add.cpp](#).

4.2.3.14 operator-()

```
template<typename T >
sparse::matrix< T > sparse::matrix< T >::operator- (
    matrix< T > const & A )
```

Operator -, subtracts the values of second matrix from the first, both must have same size and dimensions.

Parameters

<code>A</code>	A constant matrix reference used after the operator
----------------	---

Definition at line 5 of file [sparse_sub.cpp](#).

4.2.3.15 `operator=()`

```
template<typename T >
matrix< T > & matrix::operator= (
    matrix< T > const & A )
```

Operator =, creates a deep copy of the matrix and assigns it.

Parameters

<code>A</code>	A constant matrix reference used after the operator
----------------	---

Definition at line 5 of file [sparse_assignment_operator.cpp](#).

4.2.3.16 `operator==()`

```
template<typename T >
template bool sparse::matrix< T >::operator== (
    matrix< T > const & A )
```

Operator ==, compares two matrices.

Parameters

<code>A</code>	A constant matrix reference used after the operator
----------------	---

Definition at line 5 of file [sparse_comparison_operator.cpp](#).

4.2.3.17 `print2D()`

```
template<typename T >
template void sparse::matrix< T >::print2D ( )
```

Prints the values of 2D array.

Definition at line 5 of file [sparse_print2D.cpp](#).

4.2.3.18 `print_matrix()`

```
template<typename T >
template void sparse::matrix< T >::print_matrix ( )
```

Prints the sparse matrix row by row.

Definition at line 5 of file [print_matrix.cpp](#).

4.2.3.19 refresh_handle()

```
template<typename T >
void matrix::refresh_handle ( ) const
```

rebuilds a valid handle according to csr vectors.

Definition at line 5 of file [sparse_refresh_handle.cpp](#).

4.2.3.20 set_value_at()

```
template<typename T >
void matrix::set_value_at (
    size_t index,
    T value )
```

Sets value at a given index.

Parameters

<i>index</i>	The linear 0-based index in the flat values vector.
<i>value</i>	The value to set.

Definition at line 5 of file [sparse-set-value.cpp](#).

4.2.3.21 shape()

```
template<typename T >
std::pair< size_t, size_t > matrix::shape ( ) const
```

Returns the shape (dimensions) of a matrix.

Definition at line 5 of file [sparse_shape.cpp](#).

4.2.3.22 sparse_readMM()

```
template<typename T >
matrix< T > matrix::sparse_readMM (
    const std::string & MMpath ) [static]
```

Reads a Matrix Market (MM) sparse matrix, converts it from COO to CSR format and returns a CSR matrix obj (with a handle constructor)

Parameters

<i>MMpath</i>	A string with the path to the MM matrix
---------------	---

Returns

A sparse matrix object created with the handle constructor

Definition at line 6 of file [sparse_readMM.cpp](#).

4.2.3.23 `transpose()`

```
template<typename T >
matrix< T > matrix::transpose ( ) const
```

Function that transposes the matrix in 2D.

Definition at line 5 of file [sparse_transpose.cpp](#).

The documentation for this class was generated from the following files:

- [/home/johnzik/Dev/Cpp-Project/inc/dense.hpp](#)
- [/home/johnzik/Dev/Cpp-Project/inc/sparse.hpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/print_matrix.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/sparse-diagonal-getter.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/sparse-get-value.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/sparse-parentheses-operator.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/sparse-set-value.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/sparse_add.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/sparse_assignment_operator.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/sparse_banded_matrix.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/sparse_comparison_operator.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/sparse_constant_mul.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/sparse_copy_constructor.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/sparse_handle_constructor.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/sparse_identity_matrix.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/sparse_mul_sxn.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/sparse_mv.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/sparse_print2D.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/sparse_readMM.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/sparse_refresh_handle.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/sparse_shape.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/sparse_spmv.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/sparse_sub.cpp](#)
- [/home/johnzik/Dev/Cpp-Project/src/sparse_transpose.cpp](#)

Chapter 5

File Documentation

5.1 /home/johnzik/Dev/Cpp-Project/inc/cross.hpp File Reference

Header file for the cross product functions of dense and sparse matrices.

```
#include "dense.hpp"
#include "sparse.hpp"
#include <mkl.h>
Include dependency graph for cross.hpp:
```

5.2 cross.hpp

[Go to the documentation of this file.](#)

```
00001 #ifndef CROSS_HPP
00002 #define CROSS_HPP
00003
00009 #include "dense.hpp"
00010 #include "sparse.hpp"
00011 #include <mkl.h>
00012
00013 namespace dense{
00019 template<typename T>
00020 dense::matrix<T> operator*(const dense::matrix<T>& A, const sparse::matrix<T>& B); // Victor
00021 }
00022
00023 namespace sparse{
00029 template<typename T>
00030 dense::matrix<T> operator*(const sparse::matrix<T>& B, const dense::matrix<T>& A); // Victor
00031 }
00032
00033 #endif
```

5.3 /home/johnzik/Dev/Cpp-Project/inc/def.hpp File Reference

Wrapper header file for the blas functions.

```
#include <mkl.h>
#include <mkl_spblas.h>
Include dependency graph for def.hpp: This graph shows which files directly or indirectly include this file:
```

Macros

- #define [saxpy](#) cblas_saxpy
- #define [daxpy](#) cblas_daxpy
- #define [caxpy](#) cblas_caxpy
- #define [zaxpy](#) cblas_zaxpy
- #define [sscal](#) cblas_sscal
- #define [dscal](#) cblas_dscal
- #define [cscal](#) cblas_cscal
- #define [zscal](#) cblas_zscal
- #define [sgemm](#) cblas_sgemm
- #define [dgemm](#) cblas_dgemm
- #define [cgemm](#) cblas_cgemm
- #define [zgemm](#) cblas_zgemm
- #define [sgemv](#) cblas_sgemv
- #define [dgemv](#) cblas_dgemv
- #define [cgemv](#) cblas_cgemv
- #define [zgemv](#) cblas_zgemv
- #define [somatcopy](#) mkl_somatcopy
- #define [domatcopy](#) mkl_domatcopy
- #define [comatcopy](#) mkl_comatcopy
- #define [zomatcopy](#) mkl_zomatcopy
- #define [sparse_s_create_csr](#) mkl_sparse_s_create_csr
- #define [sparse_d_create_csr](#) mkl_sparse_d_create_csr
- #define [sparse_c_create_csr](#) mkl_sparse_c_create_csr
- #define [sparse_z_create_csr](#) mkl_sparse_z_create_csr
- #define [sparse_destroy](#) mkl_sparse_destroy
- #define [sparse_convert_csr](#) mkl_sparse_convert_csr
- #define [sparse_spmv](#) mkl_sparse_spmv
- #define [sparse_s_export_csr](#) mkl_sparse_s_export_csr
- #define [sparse_d_export_csr](#) mkl_sparse_d_export_csr
- #define [sparse_c_export_csr](#) mkl_sparse_c_export_csr
- #define [sparse_z_export_csr](#) mkl_sparse_z_export_csr
- #define [sparse_s_export_csr](#) mkl_sparse_s_export_csr
- #define [sparse_d_export_csr](#) mkl_sparse_d_export_csr
- #define [sparse_c_export_csr](#) mkl_sparse_c_export_csr
- #define [sparse_z_export_csr](#) mkl_sparse_z_export_csr
- #define [sparse_s_add](#) mkl_sparse_s_add
- #define [sparse_d_add](#) mkl_sparse_d_add
- #define [sparse_c_add](#) mkl_sparse_c_add
- #define [sparse_z_add](#) mkl_sparse_z_add
- #define [scsrnv](#) mkl_scsrmv
- #define [dcsrmv](#) mkl_dcsmv
- #define [ccsrmv](#) mkl_ccsrmv
- #define [zcsrmv](#) mkl_zcsrmv
- #define [sparse_s_mm](#) mkl_sparse_s_mm
- #define [sparse_d_mm](#) mkl_sparse_d_mm
- #define [sparse_c_mm](#) mkl_sparse_c_mm
- #define [sparse_z_mm](#) mkl_sparse_z_mm
- #define [sparse_s_mv](#) mkl_sparse_s_mv
- #define [sparse_d_mv](#) mkl_sparse_d_mv
- #define [sparse_c_mv](#) mkl_sparse_c_mv
- #define [sparse_z_mv](#) mkl_sparse_z_mv

5.3.1 Detailed Description

Wrapper header file for the blas functions.

Definition in file [def.hpp](#).

5.3.2 Macro Definition Documentation

5.3.2.1 caxpy

```
#define caxpy cblas_caxpy
```

Definition at line 13 of file [def.hpp](#).

5.3.2.2 ccsrmv

```
#define ccsrmv mkl_ccsrmv
```

Definition at line 66 of file [def.hpp](#).

5.3.2.3 cgemm

```
#define cgemm cblas_cgemm
```

Definition at line 23 of file [def.hpp](#).

5.3.2.4 cgemv

```
#define cgemv cblas_cgemv
```

Definition at line 28 of file [def.hpp](#).

5.3.2.5 comatcopy

```
#define comatcopy mkl_comatcopy
```

Definition at line 33 of file [def.hpp](#).

5.3.2.6 cscal

```
#define cscal cblas_cscal
```

Definition at line 18 of file [def.hpp](#).

5.3.2.7 daxpy

```
#define daxpy cblas_daxpy
```

Definition at line 12 of file [def.hpp](#).

5.3.2.8 dcsrmv

```
#define dcsrmv mkl_dcsrmv
```

Definition at line 65 of file [def.hpp](#).

5.3.2.9 dgemm

```
#define dgemm cblas_dgemm
```

Definition at line 22 of file [def.hpp](#).

5.3.2.10 dgemv

```
#define dgemv cblas_dgemv
```

Definition at line 27 of file [def.hpp](#).

5.3.2.11 domatcopy

```
#define domatcopy mkl_domatcopy
```

Definition at line 32 of file [def.hpp](#).

5.3.2.12 dscal

```
#define dscal cblas_dscal
```

Definition at line 17 of file [def.hpp](#).

5.3.2.13 saxpy

```
#define saxpy cblas_saxpy
```

Definition at line 11 of file [def.hpp](#).

5.3.2.14 scsrmv

```
#define scsrmv mkl_scsrmv
```

Definition at line 64 of file [def.hpp](#).

5.3.2.15 sgemm

```
#define sgemm cblas_sgemm
```

Definition at line 21 of file [def.hpp](#).

5.3.2.16 sgemv

```
#define sgemv cblas_sgemv
```

Definition at line 26 of file [def.hpp](#).

5.3.2.17 somatcopy

```
#define somatcopy mkl_somatcopy
```

Definition at line 31 of file [def.hpp](#).

5.3.2.18 sparse_c_add

```
#define sparse_c_add mkl_sparse_c_add
```

Definition at line 61 of file [def.hpp](#).

5.3.2.19 sparse_c_create_csr

```
#define sparse_c_create_csr mkl_sparse_c_create_csr
```

Definition at line 40 of file [def.hpp](#).

5.3.2.20 sparse_c_export_csr [1/2]

```
#define sparse_c_export_csr mkl_sparse_c_export_csr
```

Definition at line 51 of file [def.hpp](#).

5.3.2.21 sparse_c_export_csr [2/2]

```
#define sparse_c_export_csr mkl_sparse_c_export_csr
```

Definition at line 51 of file [def.hpp](#).

5.3.2.22 sparse_c_mm

```
#define sparse_c_mm mkl_sparse_c_mm
```

Definition at line 71 of file [def.hpp](#).

5.3.2.23 `sparse_c_mv`

```
#define sparse_c_mv mkl_sparse_c_mv
```

Definition at line 76 of file [def.hpp](#).

5.3.2.24 `sparse_convert_csr`

```
#define sparse_convert_csr mkl_sparse_convert_csr
```

Definition at line 45 of file [def.hpp](#).

5.3.2.25 `sparse_d_add`

```
#define sparse_d_add mkl_sparse_d_add
```

Definition at line 60 of file [def.hpp](#).

5.3.2.26 `sparse_d_create_csr`

```
#define sparse_d_create_csr mkl_sparse_d_create_csr
```

Definition at line 39 of file [def.hpp](#).

5.3.2.27 `sparse_d_export_csr` [1/2]

```
#define sparse_d_export_csr mkl_sparse_d_export_csr
```

Definition at line 50 of file [def.hpp](#).

5.3.2.28 `sparse_d_export_csr` [2/2]

```
#define sparse_d_export_csr mkl_sparse_d_export_csr
```

Definition at line 50 of file [def.hpp](#).

5.3.2.29 `sparse_d_mm`

```
#define sparse_d_mm mkl_sparse_d_mm
```

Definition at line 70 of file [def.hpp](#).

5.3.2.30 `sparse_d_mv`

```
#define sparse_d_mv mkl_sparse_d_mv
```

Definition at line 75 of file [def.hpp](#).

5.3.2.31 `sparse_destroy`

```
#define sparse_destroy mkl_sparse_destroy
```

Definition at line 43 of file [def.hpp](#).

5.3.2.32 `sparse_s_add`

```
#define sparse_s_add mkl_sparse_s_add
```

Definition at line 59 of file [def.hpp](#).

5.3.2.33 `sparse_s_create_csr`

```
#define sparse_s_create_csr mkl_sparse_s_create_csr
```

Definition at line 38 of file [def.hpp](#).

5.3.2.34 `sparse_s_export_csr` [1/2]

```
#define sparse_s_export_csr mkl_sparse_s_export_csr
```

Definition at line 49 of file [def.hpp](#).

5.3.2.35 `sparse_s_export_csr` [2/2]

```
#define sparse_s_export_csr mkl_sparse_s_export_csr
```

Definition at line 49 of file [def.hpp](#).

5.3.2.36 `sparse_s_mm`

```
#define sparse_s_mm mkl_sparse_s_mm
```

Definition at line 69 of file [def.hpp](#).

5.3.2.37 `sparse_s_mv`

```
#define sparse_s_mv mkl_sparse_s_mv
```

Definition at line 74 of file [def.hpp](#).

5.3.2.38 `sparse_spmv`

```
#define sparse_spmv mkl_sparse_spmv
```

Definition at line 47 of file [def.hpp](#).

5.3.2.39 `sparse_z_add`

```
#define sparse_z_add mkl_sparse_z_add
```

Definition at line 62 of file [def.hpp](#).

5.3.2.40 `sparse_z_create_csr`

```
#define sparse_z_create_csr mkl_sparse_z_create_csr
```

Definition at line 41 of file [def.hpp](#).

5.3.2.41 `sparse_z_export_csr` [1/2]

```
#define sparse_z_export_csr mkl_sparse_z_export_csr
```

Definition at line 52 of file [def.hpp](#).

5.3.2.42 `sparse_z_export_csr` [2/2]

```
#define sparse_z_export_csr mkl_sparse_z_export_csr
```

Definition at line 52 of file [def.hpp](#).

5.3.2.43 `sparse_z_mm`

```
#define sparse_z_mm mkl_sparse_z_mm
```

Definition at line 72 of file [def.hpp](#).

5.3.2.44 `sparse_z_mv`

```
#define sparse_z_mv mkl_sparse_z_mv
```

Definition at line 77 of file [def.hpp](#).

5.3.2.45 `sscal`

```
#define sscal cblas_sscal
```

Definition at line 16 of file [def.hpp](#).

5.3.2.46 `zaxpy`

```
#define zaxpy cblas_zaxpy
```

Definition at line 14 of file [def.hpp](#).

5.3.2.47 zcsrvmv

```
#define zcsrvmv mkl_zcsrvmv
```

Definition at line 67 of file [def.hpp](#).

5.3.2.48 zgemm

```
#define zgemm cblas_zgemm
```

Definition at line 24 of file [def.hpp](#).

5.3.2.49 zgemv

```
#define zgemv cblas_zgemv
```

Definition at line 29 of file [def.hpp](#).

5.3.2.50 zomatcopy

```
#define zomatcopy mkl_zomatcopy
```

Definition at line 34 of file [def.hpp](#).

5.3.2.51 zscal

```
#define zscal cblas_zscal
```

Definition at line 19 of file [def.hpp](#).

5.4 def.hpp

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002 // Victor
00007 #include <mkl.h>
00008 #include <mkl_spblas.h>
00009
00010 // Dense blas functions
00011 #define saxpy cblas_saxpy
00012 #define daxpy cblas_daxpy
00013 #define caxpy cblas_caxpy
00014 #define zaxpy cblas_zaxpy
00015
00016 #define sscal cblas_sscal
00017 #define dscal cblas_dscal
00018 #define cscal cblas_cscal
00019 #define zscal cblas_zscal
00020
00021 #define sgemm cblas_sgemm
00022 #define dgemm cblas_dgemm
00023 #define cgemm cblas_cgemm
00024 #define zgemm cblas_zgemm
00025
00026 #define sgemv cblas_sgemv
00027 #define dgemv cblas_dgemv
```

```

00028 #define cgemv cblas_cgemv
00029 #define zgemv cblas_zgemv
00030
00031 #define somatcopy mkl_somatcopy
00032 #define domatcopy mkl_domatcopy
00033 #define comatcopy mkl_comatcopy
00034 #define zomatcopy mkl_zomatcopy
00035
00036 // Sparse functions
00037
00038 #define sparse_s_create_csr mkl_sparse_s_create_csr
00039 #define sparse_d_create_csr mkl_sparse_d_create_csr
00040 #define sparse_c_create_csr mkl_sparse_c_create_csr
00041 #define sparse_z_create_csr mkl_sparse_z_create_csr
00042
00043 #define sparse_destroy mkl_sparse_destroy
00044
00045 #define sparse_convert_csr mkl_sparse_convert_csr
00046
00047 #define sparse_spmv mkl_sparse_spmv
00048
00049 #define sparse_s_export_csr mkl_sparse_s_export_csr
00050 #define sparse_d_export_csr mkl_sparse_d_export_csr
00051 #define sparse_c_export_csr mkl_sparse_c_export_csr
00052 #define sparse_z_export_csr mkl_sparse_z_export_csr
00053
00054 #define sparse_s_export_csr mkl_sparse_s_export_csr
00055 #define sparse_d_export_csr mkl_sparse_d_export_csr
00056 #define sparse_c_export_csr mkl_sparse_c_export_csr
00057 #define sparse_z_export_csr mkl_sparse_z_export_csr
00058
00059 #define sparse_s_add mkl_sparse_s_add
00060 #define sparse_d_add mkl_sparse_d_add
00061 #define sparse_c_add mkl_sparse_c_add
00062 #define sparse_z_add mkl_sparse_z_add
00063
00064 #define scsrsv mkl_scsrsv
00065 #define dcsrsv mkl_dcscrsv
00066 #define ccscrsv mkl_ccscrsv
00067 #define zcsrsv mkl_zcsrsv
00068
00069 #define sparse_s_mm mkl_sparse_s_mm
00070 #define sparse_d_mm mkl_sparse_d_mm
00071 #define sparse_c_mm mkl_sparse_c_mm
00072 #define sparse_z_mm mkl_sparse_z_mm
00073
00074 #define sparse_s_mv mkl_sparse_s_mv
00075 #define sparse_d_mv mkl_sparse_d_mv
00076 #define sparse_c_mv mkl_sparse_c_mv
00077 #define sparse_z_mv mkl_sparse_z_mv

```

5.5 /home/johnzik/Dev/Cpp-Project/inc/dense.hpp File Reference

Header file for the matrix class for dense matrices and its associated functions.

```

#include <iostream>
#include <vector>
#include <complex>
#include <stdexcept>
#include "def.hpp"
#include "../inc/mmio.h"

```

Include dependency graph for dense.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class `dense::matrix< T >`

A class to create a dense matrix object of any number of dimensions.

Functions

- `template<typename T, typename U>`
`matrix< T > dense::operator* (U x, const matrix< T > &A)`
*Operator *, multiplies a number by a dense matrix.*
- `template<typename T>`
`std::vector< T > dense::operator* (const std::vector< T > x, const matrix< T > &A)`
*Operator *, multiplies a vector of n collumns by a dense matrix of n rows.*
- `template<typename T>`
`std::vector< T > dense::LU_solver (dense::matrix< T > &A, std::vector< T > b)`

5.5.1 Detailed Description

Header file for the matrix class for dense matrices and its associated functions.

Definition in file [dense.hpp](#).

5.5.2 Function Documentation

5.5.2.1 LU_solver()

```
template<typename T>
std::vector< T > dense::LU_solver (
    dense::matrix< T > & A,
    std::vector< T > b )
```

Definition at line 6 of file [LU_solver.cpp](#).

5.5.2.2 operator*() [1/2]

```
template<typename T>
std::vector< T > dense::operator* (
    const std::vector< T > x,
    const matrix< T > & A )
```

Operator *, multiplies a vector of n collumns by a dense matrix of n rows.

Parameters

<code>x</code>	A vector used before the operator
----------------	-----------------------------------

Template Parameters

<code>T</code>	encompasses all primitive numerical data types and the matrix type
----------------	--

Definition at line 6 of file [dense_vm_gemv.cpp](#).

5.5.2.3 operator*() [2/2]

```
template<typename T , typename U >
matrix< T > dense::operator* (
    U x,
    const matrix< T > & A )
```

Operator *, multiplies a number by a dense matrix.

Parameters

x	A numerical primitive used before the operator
---	--

Template Parameters

U	encompasses all primitive numerical data types and T, the matrix type
---	---

Definition at line 6 of file [scal_nm.cpp](#).

5.6 dense.hpp

[Go to the documentation of this file.](#)

```
00001 #ifndef DENSE_HPP
00002 #define DENSE_HPP
00003
00009 #include <iostream>
00010 #include <vector>
00011 #include <complex>
00012 #include <stdexcept>
00013 #include "def.hpp"
00014
00015 extern "C"
00016 {
00017     #include "../inc/mmio.h"
00018 }
00019
00020 namespace sparse {
00021     template<typename T>
00022     class matrix; // forward declaration
00023 }
00024
00025 namespace dense{
00031 template<typename T>
00032 class matrix {
00033     private:
00037         std::vector<T> values;
00038
00042         std::vector<size_t> dim_size;
00043
00047         std::vector<int> precomputed_strides;
00048
00049     public:
00054         matrix (){};
00061         matrix(std::vector<T> diag, int n, int m);
00062
00068         template<typename... Dims>
00069         matrix(Dims... dims){ // Vic
00070
00071             // Add dimension sizes to the dim_size vector
00072             (dim_size.push_back(dims), ...);
00073
00074             // Resizing to the size of the matrix using a fold expression and initialising to 0.0.
00075             values.resize((static_cast<size_t>(1) * ... * static_cast<size_t>(dims)));
00076
00077             // Resizing to number of dimensions
00078             precomputed_strides.resize(dim_size.size());
00079             if (!precomputed_strides.empty()) {
00080                 precomputed_strides.back() = 1; // Last dimension has stride 1
```

```

00081
00082         for (int i = precomputed_strides.size() - 2; i >= 0; --i) {
00083             precomputed_strides[i] = precomputed_strides[i + 1] * dim_size[i + 1];
00084         }
00085     }
00086 }
00087
00092     matrix (matrix&& A) noexcept
00093     : values(std::move(A.values)), dim_size(std::move(A.dim_size)),
precomputed_strides(std::move(A.precomputed_strides)) {}
00094
00099     matrix (matrix const& A); // Ioannis
00100
00105     ~matrix() = default; // Vic
00112     matrix (std::vector<T> v, std::vector<size_t> d, std::vector<int> p); //Thanasis
00113
00118     matrix I(int N); // Thanasis
00119
00124     matrix& operator=(matrix const& A); // Ioannis
00125
00130     matrix operator+(matrix const& A); // Vic
00131
00136     template<typename U = T>
00137     std::enable_if_t<std::is_same_v<U, T>, matrix<T>>
00138     operator*(const matrix<T>& A) const; // Vic
00139
00144     bool operator==(matrix const& A){
00145         if (dim_size.size() != A.dim_size.size())
00146             return false;
00147         else{
00148             for (int i = 0; i<dim_size.size(); i++){
00149                 if (dim_size[i]!=A.dim_size[i]){
00150                     return false;
00151                 }
00152             }
00153             for (int i=0; i<values.size(); i++){
00154                 if (values[i]!=A.values[i])
00155                     return false;
00156             }
00157         }
00158         return true;
00159     }
00165     matrix operator*(T x);
00166
00172     std::vector<T> operator*(const std::vector<T> x);
00173
00178     matrix operator-(matrix const& A); //Ioannis
00179
00184     matrix transpose() const; // Victor
00189     matrix operator ^(int n); //Thanasis
00194     matrix operator/(matrix const& A);
00195
00200     matrix operator|(matrix const& A);
00201
00206     T& operator() (size_t index); //Vic
00207
00212     template<typename... Indices>
00213     const T& operator() (Indices... indices);
00214
00215
00219     size_t get_size() const; //Victor
00220
00224     size_t get_nod() const; // Ioanis
00225
00230     size_t get_dim_size(size_t index) const; // Thanasis
00231
00236     T get_value_at(size_t index) const; // Thanasis
00243     void svd(matrix<T>& u, matrix<T>& s, matrix<T>& v); // Thansis
00244
00248     T* get_values_data(); // Victor
00249
00255     void set_value_at(size_t index, T value); // Ioannis
00256
00262     template<typename... Types>
00263     void set_values(Types&&... values){
00264         std::vector<T> v;
00265         (v.push_back(values), ...);
00266         values = std::move(v);
00267     }
00268
00272     const T* get_const_values_data() const;
00273
00277     void print2D();
00278
00284     template<typename... Dims>
00285     void reshape(Dims... dims){
00286         //Clear previous dimensions size

```

```

00287         dim_size.clear();
00288         // Add dimension sizes to the dim_size vector
00289         (dim_size.push_back(dims), ...);
00290
00291         // Resizing to the size of the matrix using a fold expression and initialising to 0.0.
00292         values.resize((static_cast<size_t>(1) * ... * static_cast<size_t>(dims)));
00293
00294         // Resizing to number of dimensions
00295         precomputed_strides.resize(dim_size.size());
00296         if (!precomputed_strides.empty()) {
00297             precomputed_strides.back() = 1; // Last dimension has stride 1
00298
00299             for (int i = precomputed_strides.size() - 2; i >= 0; --i) {
00300                 precomputed_strides[i] = precomputed_strides[i + 1] * dim_size[i + 1];
00301             }
00302         }
00303     };
00304
00309     void low_level_matrix_multiplication(const CBLAS_TRANSPOSE transa, const CBLAS_TRANSPOSE
transb, const T beta, matrix<T> const &A, matrix<T> const &B, matrix<T> &C, const T alpha);
00310
00317     static matrix dense_readMM(const std::string& Mmpath);
00318 };
00319
00325     template<typename T, typename U>
00326     matrix<T> operator*(U x, const matrix<T>& A); // Victor
00327
00333     template<typename T>
00334     std::vector<T> operator*(const std::vector<T> x, const matrix<T>& A); // John
00335     template<typename T>
00336     std::vector<T> LU_solver (dense::matrix<T>& A, std::vector<T> b); // Thanasis
00337 }
00338 #endif

```

5.7 /home/johnzik/Dev/Cpp-Project/inc/matrix.hpp File Reference

Header that connects all header files.

```

#include "dense.hpp"
#include "sparse.hpp"
#include "cross.hpp"

```

Include dependency graph for matrix.hpp: This graph shows which files directly or indirectly include this file:

5.7.1 Detailed Description

Header that connects all header files.

Definition in file [matrix.hpp](#).

5.8 matrix.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef MATRIX_HPP
00002 #define MATRIX_HPP
00003
00009 #include "dense.hpp"
00010 #include "sparse.hpp"
00011 #include "cross.hpp"
00012
00013 #endif // MATRIX_HPP

```

5.9 mmio.h

```

00001 /*
00002  *   Matrix Market I/O library for ANSI C
00003  *
00004  *   See http://math.nist.gov/MatrixMarket for details.
00005  *
00006  *
00007  */
00008
00009 #ifndef MM_IO_H
00010 #define MM_IO_H
00011
00012 #define MM_MAX_LINE_LENGTH 1025
00013 #define MatrixMarketBanner "%MatrixMarket"
00014 #define MM_MAX_TOKEN_LENGTH 64
00015
00016 typedef char MM_typecode[4];
00017
00018 char *mm_typecode_to_str(MM_typecode matcode);
00019
00020 int mm_read_banner(FILE *f, MM_typecode *matcode);
00021 int mm_read_mtx_crd_size(FILE *f, int *M, int *N, int *nz);
00022 int mm_read_mtx_array_size(FILE *f, int *M, int *N);
00023
00024 int mm_write_banner(FILE *f, MM_typecode matcode);
00025 int mm_write_mtx_crd_size(FILE *f, int M, int N, int nz);
00026 int mm_write_mtx_array_size(FILE *f, int M, int N);
00027
00028
00029 /***** MM_typecode query fucntions *****/
00030
00031 #define mm_is_matrix(typecode) ((typecode)[0]=='M')
00032
00033 #define mm_is_sparse(typecode) ((typecode)[1]=='C')
00034 #define mm_is_coordinate(typecode) ((typecode)[1]=='C')
00035 #define mm_is_dense(typecode) ((typecode)[1]=='A')
00036 #define mm_is_array(typecode) ((typecode)[1]=='A')
00037
00038 #define mm_is_complex(typecode) ((typecode)[2]=='C')
00039 #define mm_is_real(typecode) ((typecode)[2]=='R')
00040 #define mm_is_pattern(typecode) ((typecode)[2]=='P')
00041 #define mm_is_integer(typecode) ((typecode)[2]=='I')
00042
00043 #define mm_is_symmetric(typecode) ((typecode)[3]=='S')
00044 #define mm_is_general(typecode) ((typecode)[3]=='G')
00045 #define mm_is_skew(typecode) ((typecode)[3]=='K')
00046 #define mm_is_hermitian(typecode) ((typecode)[3]=='H')
00047
00048 int mm_is_valid(MM_typecode matcode); /* too complex for a macro */
00049
00050
00051 /***** MM_typecode modify fucntions *****/
00052
00053 #define mm_set_matrix(typecode) ((typecode)[0]='M')
00054 #define mm_set_coordinate(typecode) ((typecode)[1]='C')
00055 #define mm_set_array(typecode) ((typecode)[1]='A')
00056 #define mm_set_dense(typecode) mm_set_array(typecode)
00057 #define mm_set_sparse(typecode) mm_set_coordinate(typecode)
00058
00059 #define mm_set_complex(typecode) ((typecode)[2]='C')
00060 #define mm_set_real(typecode) ((typecode)[2]='R')
00061 #define mm_set_pattern(typecode) ((typecode)[2]='P')
00062 #define mm_set_integer(typecode) ((typecode)[2]='I')
00063
00064
00065 #define mm_set_symmetric(typecode) ((typecode)[3]='S')
00066 #define mm_set_general(typecode) ((typecode)[3]='G')
00067 #define mm_set_skew(typecode) ((typecode)[3]='K')
00068 #define mm_set_hermitian(typecode) ((typecode)[3]='H')
00069
00070 #define mm_clear_typecode(typecode) ((typecode)[0]=(typecode)[1]= \
00071                                     (typecode)[2]=' ', (typecode)[3]='G')
00072
00073 #define mm_initialize_typecode(typecode) mm_clear_typecode(typecode)
00074
00075
00076 /***** Matrix Market error codes *****/
00077
00078
00079 #define MM_COULD_NOT_READ_FILE 11
00080 #define MM_PREMATURE_EOF 12
00081 #define MM_NOT_MTX 13
00082 #define MM_NO_HEADER 14
00083 #define MM_UNSUPPORTED_TYPE 15
00084 #define MM_LINE_TOO_LONG 16
00085 #define MM_COULD_NOT_WRITE_FILE 17

```

```

00086
00087
00088 /***** Matrix Market internal definitions *****/
00089
00090     MM_matrix_typecode: 4-character sequence
00091
00092             object      sparse/      data      storage
00093             dense      type      scheme
00094
00095     string position:  [0]      [1]      [2]      [3]
00096
00097     Matrix typecode:  M(atrix)  C(oord)  R(eal)  G(eneral)
00098                     A(array)  C(omplex) H(ermitian)
00099                     P(attern) S(ymmetric)
00100                     I(nTEGER) K(ew)
00101
00102     *****/
00103
00104 #define MM_MTX_STR      "matrix"
00105 #define MM_ARRAY_STR    "array"
00106 #define MM_DENSE_STR    "array"
00107 #define MM_COORDINATE_STR "coordinate"
00108 #define MM_SPARSE_STR   "coordinate"
00109 #define MM_COMPLEX_STR  "complex"
00110 #define MM_REAL_STR     "real"
00111 #define MM_INT_STR      "integer"
00112 #define MM_GENERAL_STR  "general"
00113 #define MM_SYMM_STR     "symmetric"
00114 #define MM_HERM_STR     "hermitian"
00115 #define MM_SKEW_STR     "skew-symmetric"
00116 #define MM_PATTERN_STR  "pattern"
00117
00118
00119 /* high level routines */
00120
00121 int mm_write_mtx_crd(char fname[], int M, int N, int nz, int I[], int J[],
00122     double val[], MM_typecode matcode);
00123 int mm_read_mtx_crd_data(FILE *f, int M, int N, int nz, int I[], int J[],
00124     double val[], MM_typecode matcode);
00125 int mm_read_mtx_crd_entry(FILE *f, int *I, int *J, double *real, double *img,
00126     MM_typecode matcode);
00127
00128 int mm_read_unsymmetric_sparse(const char *fname, int *M_, int *N_, int *nz_,
00129     double **val_, int **I_, int **J_);
00130
00131
00132
00133 #endif

```

5.10 /home/johnzik/Dev/Cpp-Project/inc/sparse.hpp File Reference

Header file for the matrix class for sparse matrices and its associated functions.

```

#include <cstring>
#include <iostream>
#include <vector>
#include <complex>
#include <stdexcept>
#include "def.hpp"
#include "../inc/mmio.h"

```

Include dependency graph for sparse.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class `sparse::matrix< T >`
A class to create a sparse matrix object in csr form.

Functions

- `template<typename T, typename U >`
`matrix< T > sparse::operator* (U x, const matrix< T > &A)`
*Operator *, multiplies a number by a sparse matrix.*
- `template<typename T >`
`std::vector< T > sparse::operator* (const std::vector< T > x, const matrix< T > &A)`
*Operator *, multiplies a vector by a sparse matrix.*
- `std::vector< double > FGMRES_dsolve (sparse::matrix< double > &A, std::vector< double > &b, std::vector< double > &x0, bool preconditioned, sparse::matrix< double > &M)`
- `std::vector< double > CG_dsolve (sparse::matrix< double > &A, std::vector< double > &b, std::vector< double > &x0)`
Double Conjugate Gradient solver for a single right-hand side (SRHS) system.

5.10.1 Detailed Description

Header file for the matrix class for sparse matrices and its associated functions.

Definition in file [sparse.hpp](#).

5.10.2 Function Documentation

5.10.2.1 CG_dsolve()

```
std::vector< double > CG_dsolve (
    sparse::matrix< double > & A,
    std::vector< double > & b,
    std::vector< double > & x0 )
```

Double Conjugate Gradient solver for a single right-hand side (SRHS) system.

Parameters

<i>A</i>	SPD Coefficient Matrix
<i>b</i>	RHS vector
<i>x0</i>	Initial guess vector (usually its set to 0)

Initialize the solver Advanced users can skip this step and set the values in the ipar and dpar arrays directly.

Check the consistency and correctness of the parameters to be passed to the solver routine. Only reduces the chance of making a mistake. Skip this operation only if you are sure that the correct data is specified in the solver parameters.

Definition at line 5 of file [CG_dsolve.cpp](#).

5.10.2.2 FGMRES_dsolve()

```
std::vector< double > FGMRES_dsolve (
    sparse::matrix< double > & A,
```

```

std::vector< double > & b,
std::vector< double > & x0,
bool preconditioned,
sparse::matrix< double > & M )

```

Definition at line 4 of file [FGMRES_dsolve.cpp](#).

5.10.2.3 operator*() [1/2]

```

template<typename T >
std::vector< T > sparse::operator* (
    const std::vector< T > x,
    const matrix< T > & A )

```

Operator *, multiplies a vector by a sparse matrix.

Parameters

<i>x</i>	A vector used before the operator
----------	-----------------------------------

Template Parameters

<i>T</i>	encompasses all primitive numerical data types and the matrix type
----------	--

Definition at line 6 of file [sparse_vxs.cpp](#).

5.10.2.4 operator*() [2/2]

```

template<typename T , typename U >
matrix< T > sparse::operator* (
    U x,
    const matrix< T > & A )

```

Operator *, multiplies a number by a sparse matrix.

Parameters

<i>x</i>	A numerical primitive used before the operator
----------	--

Template Parameters

<i>U</i>	encompasses all primitive numerical data types and T, the matrix type
----------	---

Definition at line 6 of file [sparse_mul_nxs.cpp](#).

5.11 sparse.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef SPARSE_HPP
00002 #define SPARSE_HPP
00003 #include <cstring>
00004
00010 #include <iostream>
00011 #include <vector>
00012 #include <complex>
00013 #include <stdexcept>
00014 #include "def.hpp"
00015
00016 extern "C"
00017 {
00018     #include "../inc/mmio.h"
00019 }
00020
00021 namespace dense {
00022     template<typename T>
00023     class matrix; // forward declaration
00024 }
00025
00026 namespace sparse{
00032 template <typename T>
00033 class matrix
00034 {
00035     private:
00036
00040         std::vector<T> values;
00041
00045         std::vector<MKL_INT> col_idx;
00046
00050         std::vector<MKL_INT> row_idx;
00051
00055         size_t row_size;
00056
00060         size_t col_size;
00061
00065         mutable sparse_matrix_t handle;
00066
00070         mutable bool valid_handle;
00071
00075         bool ownership;
00076
00077     public:
00081         matrix() = default;
00085         void refresh_handle() const; //Thanasis
00094         template<typename U>
00095         matrix(const std::vector<T>& values, const std::vector<U>& col_idx, const std::vector<U>&
row_idx, size_t rows, size_t cols)
00096             : values(values), col_idx(col_idx), row_idx(row_idx), row_size(rows), col_size(cols),
ownership(true), valid_handle(false), handle(nullptr)
00097             {}; //Thanasis
00098
00106         matrix(const sparse_matrix_t h, size_t rows, size_t cols, bool own); //Thanasis
00107
00111         ~matrix(){ // Victor
00112             if (valid_handle) {
00113                 mkl_sparse_destroy(handle);
00114                 valid_handle = false;
00115             };
00116
00121         matrix(matrix&& A) noexcept // Thanasis and Victor
00122             : values(std::move(A.values)), col_idx(std::move(A.col_idx)),
row_idx(std::move(A.row_idx)),
00123             row_size(A.row_size), col_size(A.col_size), handle(A.handle),
valid_handle(A.valid_handle), ownership(A.ownership){
00124             A.row_size = 0;
00125             A.col_size = 0;
00126             A.handle = nullptr;
00127             A.valid_handle = false;
00128             A.ownership = false;
00129         }
00130
00135         matrix (matrix const& A); // John
00136
00141         static matrix I(int N); // John
00142
00150         static matrix banded(int N, int kl, int ku, T val);
00151
00152         //operators
00153
00158         matrix& operator=(matrix const& A); // John
00159
00164         matrix operator+(matrix const& A); //Thanasis
00165
00170         matrix operator*(matrix const& A); // Thanasis
00171
00177         matrix operator*(T x); // Victor

```

```

00178
00184         std::vector<T> operator*(const std::vector<T> x); // John
00185
00190         matrix operator-(matrix const& A); // Thanasis
00191
00196         bool operator==(matrix const& A); // John
00197
00198         //getters-setters
00203         T get_value_at(size_t index) const; // Thanasis
00204
00210         void set_value_at(size_t index, T value); // Thanasis
00211
00217         const T operator() (size_t r, size_t c) const; // Thanasis
00218
00222         std::pair<size_t, size_t> shape() const; // Victor
00223
00224         //sparse specific (for solvers etc)
00228         std::vector<T> diag() const; // Thanasis
00229
00233         void print2D(); // John
00237         void print_matrix(); // Thanasis
00238
00242         const sparse_matrix_t& get_handle() const { return handle; } // Victor
00243
00247         void invalidate_handle() { handle = nullptr; valid_handle = false; } // Victor
00248
00252         const std::vector<T> get_values_data_const() const { return values; } // For reading Victor
00253
00257         std::vector<T>& get_values_data() { return values; } // For writing Victor
00258
00262         matrix transpose() const; // Victor
00263
00270         static matrix sparse_readMM(const std::string& MMpath);
00271
00272     };
00278     template<typename T, typename U>
00279     matrix<T> operator*(U x, const matrix<T>& A); // Victor
00280
00286     template<typename T>
00287     std::vector<T> operator*(const std::vector<T> x, const matrix<T>& A); // Victor
00288     }
00289
00290     std::vector<double> FGMRES_solve(sparse::matrix<double> & A, std::vector<double> & b,
00291                                     std::vector<double> & x0, bool preconditioned, sparse::matrix<double> & M);
00292
00298     std::vector<double> CG_solve(sparse::matrix<double> & A, std::vector<double> & b, std::vector<double>
00299                                   & x0);
00299
00300 #endif

```

5.12 axpy_add.cpp

```

00001 #include "dense.hpp"
00002 using namespace dense;
00003
00004 template <typename T>
00005 matrix<T> matrix<T>::operator+(matrix<T> const& A){
00006     matrix<T> B = *this;
00007
00008     if constexpr (std::is_same_v<T, float>){
00009         float a = 1;
00010         saxpy(static_cast<int>(A.values.size()), a, A.values.data(), 1, B.values.data(), 1);}
00011     else if constexpr (std::is_same_v<T, double>){
00012         double a = 1;
00013         daxpy(static_cast<int>(A.values.size()), a, A.values.data(), 1, B.values.data(), 1);}
00014     else if constexpr (std::is_same_v<T, std::complex<float> >){
00015         std::complex<float> a(1, 0);
00016         caxpy(static_cast<int>(A.values.size()), &a, A.values.data(), 1, B.values.data(), 1);}
00017     else if constexpr (std::is_same_v<T, std::complex<double> >){
00018         std::complex<double> a(1, 0);
00019         zaxpy(static_cast<int>(A.values.size()), &a, A.values.data(), 1, B.values.data(), 1);}
00020     return B;
00021 }
00022 template class dense::matrix<float>;
00023 template class dense::matrix<double>;
00024 template class dense::matrix<std::complex<float>>;
00025 template class dense::matrix<std::complex<double>>;

```

5.13 axpy_sub.cpp

```

00001 #include "dense.hpp"

```

```

00002 using namespace dense;
00003
00004 template <typename T>
00005 matrix<T> matrix<T>:: operator-(matrix<T> const& A){
00006     matrix<T> B = *this;
00007
00008     if constexpr (std::is_same_v<T, float>){
00009         float a = -1;
00010         saxpy(static_cast<int>(A.values.size()), a, A.values.data(), 1, B.values.data(), 1);
00011     else if constexpr (std::is_same_v<T, double>){
00012         double a = -1;
00013         daxpy(static_cast<int>(A.values.size()), a, A.values.data(), 1, B.values.data(), 1);
00014     else if constexpr (std::is_same_v<T, std::complex<float>){
00015         std::complex<float> a(-1, 0);
00016         caxpy(static_cast<int>(A.values.size()), &a, A.values.data(), 1, B.values.data(), 1);
00017     else if constexpr (std::is_same_v<T, std::complex<double>){
00018         std::complex<double> a(-1, 0);
00019         zaxpy(static_cast<int>(A.values.size()), &a, A.values.data(), 1, B.values.data(), 1);
00020     return B;
00021 }
00022 template class dense::matrix<float>;
00023 template class dense::matrix<double>;
00024 template class dense::matrix<std::complex<float>>;
00025 template class dense::matrix<std::complex<double>>;

```

5.14 CG_dsolve.cpp

```

00001 #include "../inc/sparse.hpp"
00002 #include <cstring>
00003 using namespace sparse;
00004
00005 std::vector<double> CG_dsolve(matrix<double> & A, std::vector<double> & b, std::vector<double> & x0)
00006 {
00007     double tol = 1e-12;
00008     std::vector<double> x = x0;
00009     size_t row = A.shape().first;
00010     size_t col = A.shape().second;
00011
00012     if (row!=col){
00013         throw std::invalid_argument("Can only solve square matrices");
00014     }
00015
00016     // Initialize CG parameters
00017     MKL_INT n = (MKL_INT) row;
00018     MKL_INT RCI_request;
00019     MKL_INT ipar[128];
00020     double dpar[128];
00021     std::vector<double> tmp(n*4); // Double array of size (n*4) for SRHS
00022                                 // We must store tmp on the heap to avoid stack overflow
00023
00028     dcg_init(&n, x.data(), b.data(), &RCI_request, ipar, dpar, tmp.data());
00029
00035     dcg_check(&n, x.data(), b.data(), &RCI_request, ipar, dpar, tmp.data());
00036
00037     int iter;
00038     dpar[0] = tol;
00039
00040     // Pre-allocate the vec for RCI_request == 1 for efficiency (to avoid re-allocation in loop)
00041     std::vector<double> vec(n);
00042     while (1)
00043     {
00044         dcg(&n, x.data(), b.data(), &RCI_request, ipar, dpar, tmp.data());
00045
00046         if (RCI_request == -1)
00047         {
00048             std::cout << "Maximum number of iterations was reached, but the relative stopping criterion
was not met." << std::endl;
00049             break;
00050         }
00051         else if (RCI_request == -2)
00052         {
00053             std::cout << "Attempted to divide by zero!" << std::endl;
00054             break;
00055         }
00056         else if (RCI_request == 1)
00057         {
00058             memcpy(vec.data(), tmp.data(), n*sizeof(double)); // copy temp to vec
00059             memcpy(tmp.data() + n, (A*vec).data(), n*sizeof(double)); // put the result in the
tmp[n:2*n - 1]
00060             continue;
00061         }
00062         else if (RCI_request == 0)
00063         {

```

```

00064         std::cout << "Converged succesfully!" << std::endl;
00065         break;
00066     }
00067 }
00068 dcg_get(&n , x.data(), b.data(), &RCI_request, ipar, dpar, tmp.data(), &iter);
00069 return x;
00070 }

```

5.15 copy_constructor.cpp

```

00001 #include <../inc/dense.hpp>
00002 using namespace dense;
00003
00004 template<typename T>
00005 matrix<T>::matrix(matrix<T> const& A)
00006     : values(A.values), dim_size(A.dim_size), precomputed_strides(A.precomputed_strides)
00007 {
00008 }
00009 }
00010
00011 template class dense::matrix<float>;
00012 template class dense::matrix<double>;
00013 template class dense::matrix<std::complex<float>>;
00014 template class dense::matrix<std::complex<double>>;

```

5.16 dense_AssignmentOP.cpp

```

00001 #include <../inc/dense.hpp>
00002 using namespace dense;
00003
00004 template<typename T>
00005 matrix<T>& matrix<T>::operator=(matrix<T> const& A)
00006 {
00007     values = A.values;
00008     dim_size = A.dim_size;
00009     precomputed_strides = A.precomputed_strides;
00010
00011     return *this;
00012 }
00013
00014 template class dense::matrix<float>;
00015 template class dense::matrix<double>;
00016 template class dense::matrix<std::complex<float>>;
00017 template class dense::matrix<std::complex<double>>;

```

5.17 dense_diagonal_construct.cpp

```

00001 #include " ../inc/dense.hpp"
00002 using namespace dense;
00003
00004 template<typename T>
00005 matrix<T>::matrix(std::vector<T> diag, int n, int m){
00006     values = std::vector<T> (n*m, 0);
00007     precomputed_strides = {m,1};
00008     dim_size = {static_cast<size_t>(n),static_cast<size_t>(m)};
00009
00010     for (int i=0; i<n;i++){
00011         values[i*m+i] = diag[i];
00012     }
00013 }
00014
00015 template class dense::matrix<float>;
00016 template class dense::matrix<double>;
00017 template class dense::matrix<std::complex<float>>;
00018 template class dense::matrix<std::complex<double>>;

```

5.18 dense_readMM.cpp

```

00001 #include " ../inc/dense.hpp"
00002
00003 using namespace dense;

```

```

00004
00005 template <typename T>
00006 matrix<T> matrix<T>::dense_readMM(const std::string& MMpath)
00007 {
00008     FILE *f;
00009     MM_typecode matcode;
00010     int MMrows, MMcols;
00011
00012     // Open the file
00013     if ((f=fopen(MMpath.c_str(), "r")) == NULL)
00014     {
00015         fclose(f);
00016         throw std::runtime_error("Error: Could not open file " + MMpath);
00017     }
00018
00019     // Read Banner
00020     if (mm_read_banner(f, &matcode) != 0)
00021     {
00022         fclose(f);
00023         throw std::runtime_error("Error: Could not process Matrix Market banner.");
00024     }
00025
00026     // Check if it's a valid sparse matrix
00027     if (!mm_is_matrix(matcode) || !mm_is_array(matcode))
00028     {
00029         fclose(f);
00030         throw std::runtime_error("Error: This code only supports dense array matrices.");
00031     }
00032
00033     // Read Size (Rows, Cols)
00034     if (mm_read_mtx_array_size(f, &MMrows, &MMcols) != 0)
00035     {
00036         fclose(f);
00037         throw std::runtime_error("Error: Could not read matrix dimensions.");
00038     }
00039
00040     std::cout << "Successfully opened " << MMpath << std::endl;
00041     std::cout << "Rows: " << MMrows << ", Cols: " << MMcols << std::endl;
00042
00043     // Create the dimension matrix (with 0 values)
00044     // Save the dims reversed since MM uses Col-Major order
00045     matrix<T> A(MMcols, MMrows);
00046
00047     if constexpr (std::is_same_v<T, float>)
00048     {
00049         for (int i = 0; i < (MMcols*MMrows); i++)
00050         {
00051             float val;
00052             if(fscanf(f, "%f\n", &val) != 1)
00053             {
00054                 fclose(f);
00055                 throw std::runtime_error("Error: File ended unexpectedly at line " +
00056 std::to_string(i));
00057             }
00058             A.set_value_at(i, val);
00059         }
00060     }
00061     else if constexpr (std::is_same_v<T, double>)
00062     {
00063         for (int i = 0; i < (MMcols*MMrows); i++)
00064         {
00065             double val;
00066             if(fscanf(f, "%lg\n", &val) != 1)
00067             {
00068                 fclose(f);
00069                 throw std::runtime_error("Error: File ended unexpectedly at line " +
00070 std::to_string(i));
00071             }
00072             A.set_value_at(i, val);
00073         }
00074     }
00075     else if constexpr (std::is_same_v<T, std::complex<float>>)
00076     {
00077         for (int i = 0; i < (MMcols*MMrows); i++)
00078         {
00079             float rVal, iVal;
00080             if(fscanf(f, "%f %f\n", &rVal, &iVal) != 2)
00081             {
00082                 fclose(f);
00083                 throw std::runtime_error("Error: File ended unexpectedly at line " +
00084 std::to_string(i));
00085             }
00086             std::complex<float> val(rVal, iVal);
00087             A.set_value_at(i, val);

```

```

00088     }
00089   }
00090   else if constexpr(std::is_same_v<T, std::complex<double> >)
00091   {
00092     for (int i = 0; i < (MMcols*MMrows); i++)
00093     {
00094       double rVal, iVal;
00095       if(fscanf(f, "%lg %lg\n", &rVal, &iVal) != 2)
00096       {
00097         fclose(f);
00098         throw std::runtime_error("Error: File ended unexpectedly at line " +
std::to_string(i));
00099       }
00100
00101       std::complex<double> val(rVal, iVal);
00102       A.set_value_at(i, val);
00103     }
00104   }
00105
00106   fclose(f);
00107
00108   // MM stores dense matrices in Col-Major order so for Row-Major we must return the transpose
00109   return A.transpose();
00110 }
00111
00112 template class dense::matrix<float>;
00113 template class dense::matrix<double>;
00114 template class dense::matrix<std::complex<float>>;
00115 template class dense::matrix<std::complex<double>>;

```

5.19 dense_vm_gemv.cpp

```

00001 #include "../inc/dense.hpp"
00002
00003 namespace dense
00004 {
00005   template<typename T>
00006   std::vector<T> operator*(const std::vector<T> x, const matrix<T>& A)
00007   {
00008     // Dimension mismatch check
00009     if (A.get_nod() < 2) {
00010       throw std::runtime_error("v*A requires A to have at least 2 dimensions.");
00011     }
00012     const MKL_INT M = static_cast<MKL_INT>(A.get_dim_size(0));
00013     const MKL_INT N = static_cast<MKL_INT>(A.get_dim_size(1));
00014
00015     if (static_cast<MKL_INT>(x.size()) != M) {
00016       throw std::runtime_error("Vector size (" + std::to_string(x.size()) +
00017                                ") does not match matrix's first dimension (" +
std::to_string(M) + ").");
00018     }
00019
00020     std::vector<T> result(A.get_dim_size(1));
00021
00022     if constexpr(std::is_same_v<T, float>)
00023     {
00024       float a = 1.0;
00025       float b = 0.0;
00026       sgemv(CblasRowMajor, CblasTrans, A.get_dim_size(0), A.get_dim_size(1), a,
A.get_const_values_data(),
00027             A.get_dim_size(1), x.data(), 1, b, result.data(), 1);
00028
00029       return result;
00030     }
00031     else if constexpr(std::is_same_v<T, double>)
00032     {
00033       double a = 1.0;
00034       double b = 0.0;
00035       dgemv(CblasRowMajor, CblasTrans, A.get_dim_size(0), A.get_dim_size(1), a,
A.get_const_values_data(),
00036             A.get_dim_size(1), x.data(), 1, b, result.data(), 1);
00037
00038       return result;
00039     }
00040     else if constexpr(std::is_same_v<T, std::complex<float> >)
00041     {
00042       std::complex<float> a(1, 0);
00043       std::complex<float> b(0, 0);
00044       cgemv(CblasRowMajor, CblasTrans, A.get_dim_size(0), A.get_dim_size(1), &a,
A.get_const_values_data(),
00045             A.get_dim_size(1), x.data(), 1, &b, result.data(), 1);
00046
00047       return result;

```



```

00048     }
00049     else if constexpr(std::is_same_v<T, std::complex<double> >)
00050     {
00051         std::complex<double> a(1, 0);
00052         std::complex<double> b(0, 0);
00053         zgemv(CblasRowMajor, CblasTrans, A.get_dim_size(0), A.get_dim_size(1), &a,
00054             A.get_const_values_data(),
00055             A.get_dim_size(1), x.data(), 1, &b, result.data(), 1);
00056         return result;
00057     }
00058 }
00059
00060 template std::vector<float> operator*(const std::vector<float> x, const dense::matrix<float>&);
00061 template std::vector<double> operator*(const std::vector<double> x, const dense::matrix<double>&);
00062 template std::vector<std::complex<float> > operator*(const std::vector<std::complex<float> > x, const
00063 dense::matrix<std::complex<float> >&);
00064 template std::vector<std::complex<double> > operator*(const std::vector<std::complex<double> > x,
00065 const dense::matrix<std::complex<double> >&);
00066 }

```

5.20 exp_operator.cpp

```

00001 #include "dense.hpp"
00002 #include <cstring>
00003 using namespace dense;
00004
00005 template <typename T>
00006 matrix<T> matrix<T>::operator ^ (int n){
00007     if (dim_size[0]!=dim_size[1] || dim_size.size()>2){
00008         throw std::invalid_argument("Can only exponentiate square 2D matrices");
00009     }
00010     if (n == 0){
00011         return matrix<T>().I(dim_size[1]);
00012     }
00013     if (n == 1){
00014         return (*this);
00015     }
00016     if (n>1){
00017         matrix<T> A = *this; //copy to alter safely without destroying *this
00018         matrix<T> res = matrix<T>().I(dim_size[1]);
00019         while (n>0){
00020             if(n&1) res = res*A;
00021             n = n>>1;
00022             A = A*A;
00023         }
00024         return res;
00025     }
00026     if (n == -1){
00027         lapack_int info;
00028         lapack_int r = dim_size[0];
00029         lapack_int * ipiv = new lapack_int[r];
00030
00031         size_t mem1 = r*r*sizeof(T);
00032         T* A_data = (T*) mkl_malloc(mem1, 64);
00033         memcpy(A_data, (*this).get_values_data(), mem1);
00034
00035         if constexpr (std::is_same_v<T, float>){
00036             info = LAPACKE_sgetrf (LAPACK_ROW_MAJOR, r, r, A_data, r, ipiv);
00037             info = LAPACKE_sgetri (LAPACK_ROW_MAJOR, r, A_data, r, ipiv);
00038         }
00039         else if constexpr (std::is_same_v<T, double>){
00040             info = LAPACKE_dgetrf (LAPACK_ROW_MAJOR, r, r, A_data, r, ipiv);
00041             info = LAPACKE_dgetri (LAPACK_ROW_MAJOR, r, A_data, r, ipiv);
00042         }
00043         else if constexpr (std::is_same_v<T, std::complex<float> >){
00044             info = LAPACKE_cgetrf (LAPACK_ROW_MAJOR, r, r, reinterpret_cast<MKL_Complex8*> (A_data), r,
00045 ipiv);
00046             info = LAPACKE_cgetri (LAPACK_ROW_MAJOR, r, reinterpret_cast<MKL_Complex8*> (A_data), r, ipiv);
00047         }
00048         else if constexpr (std::is_same_v<T, std::complex<double> >){
00049             info = LAPACKE_zgetrf (LAPACK_ROW_MAJOR, r, r, reinterpret_cast<MKL_Complex16*> (A_data), r,
00050 ipiv);
00051             info = LAPACKE_zgetri (LAPACK_ROW_MAJOR, r, reinterpret_cast<MKL_Complex16*> (A_data), r,
00052 ipiv);
00053         }
00054         std::vector<T> v(A_data, A_data+r*r);
00055         matrix<T> ret(v, dim_size, precomputed_strides);
00056         return ret;
00057     }
00058     if (n<-1){
00059         return ((*this)^(-1))^(-n);
00060     }

```

```

00058 }
00059 template class dense::matrix<float>;
00060 template class dense::matrix<double>;
00061 template class dense::matrix<std::complex<float>>;
00062 template class dense::matrix<std::complex<double>>;

```

5.21 FGMRES_dsolve.cpp

```

00001 #include "../inc/sparse.hpp"
00002 #include <cstring>
00003 using namespace sparse;
00004 std::vector<double> FGMRES_dsolve(matrix<double> & A, std::vector<double> & b, std::vector<double> &
    x0, bool preconditioned, matrix<double> &M){
00005     double epsilon = 1e-12;
00006     std::vector<double> x = x0;
00007     size_t r = A.shape().first;
00008     size_t c = A.shape().second;
00009
00010     if (r!=c){
00011         throw std::invalid_argument("Can only solve square matrices");
00012     }
00013
00014     int n = (int) r;
00015     int RCI_request;
00016     int ipar[128];
00017     double dpar[128];
00018     int tmp_size = ((2*std::min(n,150) + 1)*n + std::min(n,150) *(std::min(n,150) + 9)/2 + 1);
00019     double * tmp = (double*)mkl_malloc(tmp_size*sizeof(double),64);
00020
00021     dfgmres_init(&n, x.data(), b.data(), &RCI_request, ipar, dpar, tmp);
00022
00023     ipar[11] = 1;
00024     ipar[9] = 0;
00025     ipar[10] = (int) preconditioned;
00026     dpar[0] = epsilon;
00027
00028     dfgmres_check(&n, x.data(), b.data(), &RCI_request, ipar, dpar, tmp);
00029
00030     int itercount;
00031     std::vector<double> vec(n);
00032
00033     while (1){
00034         dfgmres(&n, x.data(), b.data(), &RCI_request, ipar, dpar, tmp);
00035         if (RCI_request == 1){
00036             memcpy(vec.data(), tmp + ipar[21]-1, n*sizeof(double));
00037             memcpy(tmp + ipar[22]-1, (A*vec).data(), n*sizeof(double));
00038
00039             continue;
00040         }
00041         else if (RCI_request == 3){
00042             memcpy(vec.data(), tmp + ipar[21]-1, n*sizeof(double));
00043             memcpy(tmp + ipar[22]-1, (M*vec).data(), n*sizeof(double));
00044
00045             continue;
00046         }
00047         if (RCI_request == 0){
00048             break;
00049         }
00050     }
00051     dfgmres_get(&n, x.data(), b.data(), &RCI_request, ipar, dpar, tmp, &itercount);
00052     return x;
00053 }

```

5.22 gemm_high.cpp

```

00001 #include "dense.hpp"
00002 using namespace dense;
00003
00004 template<typename T>
00005 template<typename U>
00006 std::enable_if_t<std::is_same_v<U, T>, matrix<T>>
00007 matrix<T>::operator*(matrix<T> const& A) const{
00008     matrix<T> B((*this).dim_size[0], A.dim_size[1]);
00009
00010     if constexpr (std::is_same_v<T, float>){
00011         float a = 1;
00012         sgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
00013             (*this).dim_size[0], A.dim_size[1],
00014             (*this).dim_size[1], a, (*this).values.data(), (*this).precomputed_strides[0],

```

```

00015         A.values.data(), A.precomputed_strides[0], 0, B.values.data(),
B.precomputed_strides[0]);}
00016     else if constexpr (std::is_same_v<T, double>){
00017         double a = 1;
00018         dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
00019             (*this).dim_size[0], A.dim_size[1],
00020             (*this).dim_size[1], a, (*this).values.data(), (*this).precomputed_strides[0],
00021             A.values.data(), A.precomputed_strides[0], 0, B.values.data(),
B.precomputed_strides[0]);}
00022     else if constexpr (std::is_same_v<T, std::complex<float>){
00023         std::complex<float> a(1, 0);
00024         std::complex<float> b(0, 0);
00025         cgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
00026             (*this).dim_size[0], A.dim_size[1],
00027             (*this).dim_size[1], &a, (*this).values.data(), (*this).precomputed_strides[0],
00028             A.values.data(), A.precomputed_strides[0], &b, B.values.data(),
B.precomputed_strides[0]);}
00029     else if constexpr (std::is_same_v<T, std::complex<double>){
00030         std::complex<double> a(1, 0);
00031         std::complex<double> b(0, 0);
00032         zgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
00033             (*this).dim_size[0], A.dim_size[1],
00034             (*this).dim_size[1], &a, (*this).values.data(), (*this).precomputed_strides[0],
00035             A.values.data(), A.precomputed_strides[0], &b, B.values.data(),
B.precomputed_strides[0]);}
00036     return B;
00037 }
00038
00039 template std::enable_if_t<std::is_same_v<float, float>, dense::matrix<float>>
00040 dense::matrix<float>::operator*<float>(const dense::matrix<float>&) const;
00041
00042 template std::enable_if_t<std::is_same_v<double, double>, dense::matrix<double>>
00043 dense::matrix<double>::operator*<double>(const dense::matrix<double>&) const;
00044
00045 template std::enable_if_t<std::is_same_v<std::complex<float>, std::complex<float>,
dense::matrix<std::complex<float>>>
00046 dense::matrix<std::complex<float>>::operator*<std::complex<float>>(const
dense::matrix<std::complex<float>>&) const;
00047
00048 template std::enable_if_t<std::is_same_v<std::complex<double>, std::complex<double>,
dense::matrix<std::complex<double>>>
00049 dense::matrix<std::complex<double>>::operator*<std::complex<double>>(const
dense::matrix<std::complex<double>>&) const;

```

5.23 gemm_low.cpp

```

00001 #include "../inc/dense.hpp"
00002 using namespace dense;
00003
00004 template<typename T>
00005 void matrix_multiplication(const CBLAS_TRANSPOSE transa, const CBLAS_TRANSPOSE transb, const T beta,
matrix<T> const& A, matrix<T> const& B, matrix<T> &C, const T alpha){
00006     if constexpr (std::is_same_v<T, float>){
00007         cblas_sgemm(CblasRowMajor,
transa, transb, (MKL_INT)A.dim_size[0], (MKL_INT)B.dim_size[1], (MKL_INT)A.dim_size[1], alpha, A.get_values_data(), (MKL_INT)A
00008         else if constexpr (std::is_same_v<T, double>){
00009             cblas_dgemm(CblasRowMajor,
transa, transb, (MKL_INT)A.dim_size[0], (MKL_INT)B.dim_size[1], (MKL_INT)A.dim_size[1], alpha, A.get_values_data(), (MKL_INT)A
00010         else if constexpr (std::is_same_v<T, std::complex<float>){
00011             cblas_cgemm(CblasRowMajor,
transa, transb, (MKL_INT)A.dim_size[0], (MKL_INT)B.dim_size[1], (MKL_INT)A.dim_size[1], reinterpret_cast<const
void*>(&alpha), A.get_values_data(), (MKL_INT)A.dim_size[1], B.get_values_data(), (MKL_INT)B.dim_size[1], reinterpret_cast<co
void*>(&beta), C.get_values_data(), (MKL_INT)C.dim_size[1]);
00012         else if constexpr (std::is_same_v<T, std::complex<double>){
00013             cblas_zgemm(CblasRowMajor,
transa, transb, (MKL_INT)A.dim_size[0], (MKL_INT)B.dim_size[1], (MKL_INT)A.dim_size[1], reinterpret_cast<const
void*>(&alpha), A.get_values_data(), (MKL_INT)A.dim_size[1], B.get_values_data(), (MKL_INT)B.dim_size[1], reinterpret_cast<co
void*>(&beta), C.get_values_data(), (MKL_INT)C.dim_size[1]);
00014     }
00015     template class dense::matrix<float>;
00016     template class dense::matrix<double>;
00017     template class dense::matrix<std::complex<float>>;
00018     template class dense::matrix<std::complex<double>>;

```

5.24 gemv.cpp

```

00001 #include "dense.hpp"
00002 using namespace dense;
00003

```

```

00004 template <typename T>
00005 std::vector<T> matrix<T>::operator*(const std::vector<T> x){
00006     std::vector<T> result((*this).dim_size[0]);
00007
00008     if constexpr (std::is_same_v<T, float>){
00009         sgemv(CblasRowMajor, CblasNoTrans, (*this).dim_size[0], (*this).dim_size[1], 1.0f,
00010             (*this).values.data(), (*this).dim_size[1], x.data(), 1, 0.0f, result.data(), 1);}
00011     else if constexpr (std::is_same_v<T, double>){
00012         dgemv(CblasRowMajor, CblasNoTrans, (*this).dim_size[0], (*this).dim_size[1], 1.0,
00013             (*this).values.data(), (*this).dim_size[1], x.data(), 1, 0.0, result.data(), 1);}
00014     else if constexpr (std::is_same_v<T, std::complex<float>){
00015         std::complex<float> a(1, 0);
00016         std::complex<float> b(0, 0);
00017         cgemv(CblasRowMajor, CblasNoTrans, (*this).dim_size[0], (*this).dim_size[1], &a,
00018             (*this).values.data(), (*this).dim_size[1], x.data(), 1, &b, result.data(), 1);}
00019     else if constexpr (std::is_same_v<T, std::complex<double>){
00020         std::complex<double> a(1, 0);
00021         std::complex<double> b(0, 0);
00022         zgemv(CblasRowMajor, CblasNoTrans, (*this).dim_size[0], (*this).dim_size[1], &a,
00023             (*this).values.data(), (*this).dim_size[1], x.data(), 1, &b, result.data(), 1);}
00024     return result;
00025 }
00026 template class dense::matrix<float>;
00027 template class dense::matrix<double>;
00028 template class dense::matrix<std::complex<float>>;
00029 template class dense::matrix<std::complex<double>>;

```

5.25 get_dim_size.cpp

```

00001 #include "../inc/dense.hpp"
00002 using namespace dense;
00003
00004 template<typename T>
00005 size_t matrix<T>::get_dim_size(size_t index) const{
00006     return dim_size[(int) index];
00007 }
00008 template size_t dense::matrix<float>::get_dim_size(size_t) const;
00009 template size_t dense::matrix<double>::get_dim_size(size_t) const;
00010 template size_t dense::matrix<std::complex<float>>::get_dim_size(size_t) const;
00011 template size_t dense::matrix<std::complex<double>>::get_dim_size(size_t) const;

```

5.26 get_nod.cpp

```

00001 #include <../inc/dense.hpp>
00002 using namespace dense;
00003
00004 template<typename T>
00005 size_t matrix<T>::get_nod() const
00006 {
00007     return dim_size.size();
00008 }
00009 // template class matrix<int>;
00010 template size_t dense::matrix<int>::get_nod() const;
00011 template size_t dense::matrix<float>::get_nod() const;
00012 template size_t dense::matrix<double>::get_nod() const;
00013 template size_t dense::matrix<std::complex<float>>::get_nod() const;
00014 template size_t dense::matrix<std::complex<double>>::get_nod() const;

```

5.27 get_size.cpp

```

00001 #include "dense.hpp"
00002 using namespace dense;
00003
00004 template<typename T>
00005 size_t matrix<T>::get_size() const{
00006     return values.size();
00007 };
00008
00009 template class dense::matrix<float>;
00010 template class dense::matrix<double>;
00011 template class dense::matrix<std::complex<float>>;
00012 template class dense::matrix<std::complex<double>>;

```

5.28 get_value_at.cpp

```

00001 #include "../inc/dense.hpp"
00002 using namespace dense;
00003
00004 template<typename T>
00005 T matrix<T>::get_value_at(size_t index) const{
00006     return values[(int)index];
00007 }
00008 template class dense::matrix<float>;
00009 template class dense::matrix<double>;
00010 template class dense::matrix<std::complex<float>>;
00011 template class dense::matrix<std::complex<double>>;

```

5.29 get_values_data.cpp

```

00001 #include "dense.hpp"
00002 using namespace dense;
00003
00004 template <typename T>
00005 T* matrix<T>::get_values_data() {
00006     return values.data();
00007 }
00008
00009 template class dense::matrix<float>;
00010 template class dense::matrix<double>;
00011 template class dense::matrix<std::complex<float>>;
00012 template class dense::matrix<std::complex<double>>;

```

5.30 get_values_data_const.cpp

```

00001 #include "../inc/dense.hpp"
00002 using namespace dense;
00003
00004 template<typename T>
00005 const T* matrix<T>::get_const_values_data() const{
00006     return values.data();
00007 }
00008 template const float* dense::matrix<float>::get_const_values_data() const;
00009 template const double* dense::matrix<double>::get_const_values_data() const;
00010 template const std::complex<float>* dense::matrix<std::complex<float>>::get_const_values_data() const;
00011 template const std::complex<double>* dense::matrix<std::complex<double>>::get_const_values_data()
00012     const;

```

5.31 identity_constructor.cpp

```

00001 #include "../inc/dense.hpp"
00002 using namespace dense;
00003
00004 template<typename T>
00005 matrix<T> matrix<T>::I(int N){
00006     matrix<T> res(N,N);
00007     for (int i=0; i<N;i++){
00008         res.get_values_data()[i*N+i] = static_cast<T>(1);
00009     }
00010     return res;
00011 }
00012 template dense::matrix<float> matrix<float>::I(int);
00013 template dense::matrix<double> matrix<double>::I(int);
00014 template dense::matrix<std::complex<float>> matrix<std::complex<float>>::I(int);
00015 template dense::matrix<std::complex<double>> matrix<std::complex<double>>::I(int);

```

5.32 low_level_multiplication.cpp

```

00001 #include <../inc/dense.hpp>
00002 using namespace dense;
00003
00004 template<typename T>
00005 void matrix<T>::low_level_matrix_multiplication(const CBLAS_TRANSPOSE transa, const CBLAS_TRANSPOSE
00006     transb, const T beta, matrix<T> const &A, matrix<T> const &B, matrix<T> &C, const T alpha){

```

```

00006     if constexpr (std::is_same_v<T, float>){
00007         cblas_sgemm(CblasRowMajor,
transa,transb,(MKL_INT)A.dim_size[0],(MKL_INT)B.dim_size[1],(MKL_INT)B.dim_size[0],alpha,A.values.data(),(MKL_INT)A.dim
00008     }
00009     else if constexpr (std::is_same_v<T,double>){
00010         cblas_dgemm(CblasRowMajor,
transa,transb,(MKL_INT)A.dim_size[0],(MKL_INT)B.dim_size[1],(MKL_INT)B.dim_size[0],alpha,A.values.data(),(MKL_INT)A.dim
00011     }
00012     else if constexpr (std::is_same_v<T, std::complex<float> >){
00013         cblas_cgemm(CblasRowMajor,
transa,transb,(MKL_INT)A.dim_size[0],(MKL_INT)B.dim_size[1],(MKL_INT)B.dim_size[0],reinterpret_cast<const
void*>(&alpha),A.values.data(),(MKL_INT)A.dim_size[1],B.values.data(),(MKL_INT)B.dim_size[1],reinterpret_cast<const
void*>(&beta),C.values.data(),(MKL_INT)C.dim_size[1]);
00014     }
00015     else if constexpr (std::is_same_v<T, std::complex<double> >){
00016         cblas_zgemm(CblasRowMajor,
transa,transb,(MKL_INT)A.dim_size[0],(MKL_INT)B.dim_size[1],(MKL_INT)B.dim_size[0],reinterpret_cast<const
void*>(&alpha),A.values.data(),(MKL_INT)A.dim_size[1],B.values.data(),(MKL_INT)B.dim_size[1],reinterpret_cast<const
void*>(&beta),C.values.data(),(MKL_INT)C.dim_size[1]);
00017     }
00018 }
00019
00020 template void dense::matrix<float>::low_level_matrix_multiplication(const CBLAS_TRANSPOSE transa,
const CBLAS_TRANSPOSE transb,const float beta, matrix<float> const& A, matrix<float> const& B,
matrix<float> &C, const float alpha);
00021 template void dense::matrix<double>::low_level_matrix_multiplication(const CBLAS_TRANSPOSE transa,
const CBLAS_TRANSPOSE transb,const double beta, matrix<double> const& A, matrix<double> const& B,
matrix<double> &C, const double alpha);
00022 template void dense::matrix<std::complex<double> >::low_level_matrix_multiplication(const
CBLAS_TRANSPOSE transa, const CBLAS_TRANSPOSE transb,const std::complex<double> beta,
matrix<std::complex<double> > const& A, matrix<std::complex<double> > const& B,
matrix<std::complex<double> > &C, const std::complex<double> alpha);
00023 template void dense::matrix<std::complex<float> >::low_level_matrix_multiplication(const
CBLAS_TRANSPOSE transa, const CBLAS_TRANSPOSE transb,const std::complex<float> beta,
matrix<std::complex<float> > const& A, matrix<std::complex<float> > const& B, matrix<std::complex<float> >
&C, const std::complex<float> alpha);

```

5.33 LU_solver.cpp

```

00001 #include "dense.hpp"
00002 #include <cstring>
00003
00004 namespace dense{
00005 template <typename T>
00006 std::vector<T> LU_solver (dense::matrix<T>& A, std::vector<T> b){
00007     if (A.get_dim_size(1)!=A.get_dim_size(0)){
00008         throw std::invalid_argument("Can only solve square matrices");
00009     }
00010     lapack_int info;
00011     lapack_int r = A.get_dim_size(0);
00012     lapack_int * ipiv = new lapack_int[r];
00013
00014     size_t mem1 = r*r*sizeof(T);
00015     T* A_data = (T*) mkl_malloc(mem1, 64);
00016     memcpy(A_data, A.get_values_data(), mem1);
00017
00018     size_t mem2 = b.size()*sizeof(T);
00019     T* b_data = (T*) mkl_malloc(mem2, 64);
00020     memcpy(b_data, b.data(), mem2);
00021
00022     if constexpr (std::is_same_v<T, float>){
00023         info = LAPACKE_sgetrf (LAPACK_ROW_MAJOR , r , r , A_data , r , ipiv);
00024         info = LAPACKE_sgetrs(LAPACK_ROW_MAJOR, 'N', r, 1, A_data, r, ipiv, b_data, 1);
00025     }
00026     else if constexpr (std::is_same_v<T, double>){
00027         info = LAPACKE_dgetrf (LAPACK_ROW_MAJOR , r , r , A_data , r , ipiv);
00028         info = LAPACKE_dgetrs(LAPACK_ROW_MAJOR, 'N', r, 1, A_data, r, ipiv, b_data, 1);
00029     }
00030     else if constexpr (std::is_same_v<T, std::complex<float> >){
00031         info = LAPACKE_cgetrf (LAPACK_ROW_MAJOR , r , r , reinterpret_cast<MKL_Complex8*> (A_data) , r ,
ipiv);
00032         info = LAPACKE_cgetrs(LAPACK_ROW_MAJOR, 'N', r, 1, reinterpret_cast<MKL_Complex8*> (A_data), r,
ipiv, reinterpret_cast<MKL_Complex8*> (b_data), 1);
00033     }
00034     else if constexpr (std::is_same_v<T, std::complex<double> >){
00035         info = LAPACKE_zgetrf (LAPACK_ROW_MAJOR , r , r , reinterpret_cast<MKL_Complex16*> (A_data) , r ,
ipiv);
00036         info = LAPACKE_zgetrs(LAPACK_ROW_MAJOR, 'N', r, 1, reinterpret_cast<MKL_Complex16*> (A_data), r,
ipiv, reinterpret_cast<MKL_Complex16*> (b_data), 1);
00037     }
00038     std::vector<T> ret(b_data, b_data + r);
00039     mkl_free(A_data);
00040     mkl_free(b_data);

```

```

00041     return ret;
00042 }
00043
00044 template<std::vector<float> LU_solver(dense::matrix<float>&, std::vector<float> b);
00045 template<std::vector<double> LU_solver(dense::matrix<double>&, std::vector<double> b);
00046 template<std::vector<std::complex<float> LU_solver(dense::matrix<std::complex<float>&,
std::vector<std::complex<float> b);
00047 template<std::vector<std::complex<double> LU_solver(dense::matrix<std::complex<double>&,
std::vector<std::complex<double> b);
00048 }

```

5.34 mmio.c

```

00001 /*
00002  * Matrix Market I/O library for ANSI C
00003  *
00004  * See http://math.nist.gov/MatrixMarket for details.
00005  *
00006  *
00007  */
00008
00009
00010 #include <stdio.h>
00011 #include <string.h>
00012 #include <stdlib.h>
00013 #include <ctype.h>
00014
00015 #include "../inc/mmio.h"
00016
00017 int mm_read_unsymmetric_sparse(const char *fname, int *M_, int *N_, int *nz_,
00018                             double **val_, int **I_, int **J_)
00019 {
00020     FILE *f;
00021     MM_typecode matcode;
00022     int M, N, nz;
00023     int i;
00024     double *val;
00025     int *I, *J;
00026
00027     if ((f = fopen(fname, "r")) == NULL)
00028         return -1;
00029
00030
00031     if (mm_read_banner(f, &matcode) != 0)
00032     {
00033         printf("mm_read_unsymetric: Could not process Matrix Market banner ");
00034         printf(" in file [%s]\n", fname);
00035         return -1;
00036     }
00037
00038
00039
00040     if ( !(mm_is_real(matcode) && mm_is_matrix(matcode) &&
00041          mm_is_sparse(matcode)) )
00042     {
00043         fprintf(stderr, "Sorry, this application does not support ");
00044         fprintf(stderr, "Market Market type: [%s]\n",
00045             mm_typecode_to_str(matcode));
00046         return -1;
00047     }
00048
00049     /* find out size of sparse matrix: M, N, nz .... */
00050
00051     if (mm_read_mtx_crd_size(f, &M, &N, &nz) !=0)
00052     {
00053         fprintf(stderr, "read_unsymmetric_sparse(): could not parse matrix size.\n");
00054         return -1;
00055     }
00056
00057     *M_ = M;
00058     *N_ = N;
00059     *nz_ = nz;
00060
00061     /* reseve memory for matrices */
00062
00063     I = (int *) malloc(nz * sizeof(int));
00064     J = (int *) malloc(nz * sizeof(int));
00065     val = (double *) malloc(nz * sizeof(double));
00066
00067     *val_ = val;
00068     *I_ = I;
00069     *J_ = J;
00070

```

```

00071      /* NOTE: when reading in doubles, ANSI C requires the use of the "l" */
00072      /* specifier as in "%lg", "%lf", "%le", otherwise errors will occur */
00073      /* (ANSI C X3.159-1989, Sec. 4.9.6.2, p. 136 lines 13-15) */
00074
00075      for (i=0; i<nz; i++)
00076      {
00077          fscanf(f, "%d %d %lg\n", &I[i], &J[i], &val[i]);
00078          I[i]--; /* adjust from 1-based to 0-based */
00079          J[i]--;
00080      }
00081      fclose(f);
00082
00083      return 0;
00084 }
00085
00086 int mm_is_valid(MM_typecode matcode)
00087 {
00088     if (!mm_is_matrix(matcode)) return 0;
00089     if (mm_is_dense(matcode) && mm_is_pattern(matcode)) return 0;
00090     if (mm_is_real(matcode) && mm_is_hermitian(matcode)) return 0;
00091     if (mm_is_pattern(matcode) && (mm_is_hermitian(matcode) ||
00092         mm_is_skew(matcode))) return 0;
00093     return 1;
00094 }
00095
00096 int mm_read_banner(FILE *f, MM_typecode *matcode)
00097 {
00098     char line[MM_MAX_LINE_LENGTH];
00099     char banner[MM_MAX_TOKEN_LENGTH];
00100     char mtx[MM_MAX_TOKEN_LENGTH];
00101     char crd[MM_MAX_TOKEN_LENGTH];
00102     char data_type[MM_MAX_TOKEN_LENGTH];
00103     char storage_scheme[MM_MAX_TOKEN_LENGTH];
00104     char *p;
00105
00106
00107     mm_clear_typecode(matcode);
00108
00109     if (fgets(line, MM_MAX_LINE_LENGTH, f) == NULL)
00110         return MM_PREMATURE_EOF;
00111
00112     if (sscanf(line, "%s %s %s %s %s", banner, mtx, crd, data_type,
00113         storage_scheme) != 5)
00114         return MM_PREMATURE_EOF;
00115
00116     for (p=mtx; *p!='\0'; *p=tolower(*p),p++); /* convert to lower case */
00117     for (p=crd; *p!='\0'; *p=tolower(*p),p++);
00118     for (p=data_type; *p!='\0'; *p=tolower(*p),p++);
00119     for (p=storage_scheme; *p!='\0'; *p=tolower(*p),p++);
00120
00121     /* check for banner */
00122     if (strcmp(banner, MatrixMarketBanner, strlen(MatrixMarketBanner)) != 0)
00123         return MM_NO_HEADER;
00124
00125     /* first field should be "mtx" */
00126     if (strcmp(mtx, MM_MTX_STR) != 0)
00127         return MM_UNSUPPORTED_TYPE;
00128     mm_set_matrix(matcode);
00129
00130
00131     /* second field describes whether this is a sparse matrix (in coordinate
00132        storgae) or a dense array */
00133
00134
00135     if (strcmp(crd, MM_SPARSE_STR) == 0)
00136         mm_set_sparse(matcode);
00137     else
00138         if (strcmp(crd, MM_DENSE_STR) == 0)
00139             mm_set_dense(matcode);
00140     else
00141         return MM_UNSUPPORTED_TYPE;
00142
00143
00144     /* third field */
00145
00146     if (strcmp(data_type, MM_REAL_STR) == 0)
00147         mm_set_real(matcode);
00148     else
00149         if (strcmp(data_type, MM_COMPLEX_STR) == 0)
00150             mm_set_complex(matcode);
00151     else
00152         if (strcmp(data_type, MM_PATTERN_STR) == 0)
00153             mm_set_pattern(matcode);
00154     else
00155         if (strcmp(data_type, MM_INT_STR) == 0)
00156             mm_set_integer(matcode);
00157     else

```



```

00158         return MM_UNSUPPORTED_TYPE;
00159
00160
00161     /* fourth field */
00162
00163     if (strcmp(storage_scheme, MM_GENERAL_STR) == 0)
00164         mm_set_general(matcode);
00165     else
00166         if (strcmp(storage_scheme, MM_SYMM_STR) == 0)
00167             mm_set_symmetric(matcode);
00168     else
00169         if (strcmp(storage_scheme, MM_HERM_STR) == 0)
00170             mm_set_hermitian(matcode);
00171     else
00172         if (strcmp(storage_scheme, MM_SKEW_STR) == 0)
00173             mm_set_skew(matcode);
00174     else
00175         return MM_UNSUPPORTED_TYPE;
00176
00177     return 0;
00178 }
00179
00180
00181 int mm_write_mtx_crd_size(FILE *f, int M, int N, int nz)
00182 {
00183     if (fprintf(f, "%d %d %d\n", M, N, nz) != 3)
00184         return MM_COULD_NOT_WRITE_FILE;
00185     else
00186         return 0;
00187 }
00188
00189 int mm_read_mtx_crd_size(FILE *f, int *M, int *N, int *nz )
00190 {
00191     char line[MM_MAX_LINE_LENGTH];
00192     int num_items_read;
00193
00194     /* set return null parameter values, in case we exit with errors */
00195     *M = *N = *nz = 0;
00196
00197     /* now continue scanning until you reach the end-of-comments */
00198     do
00199     {
00200         if (fgets(line, MM_MAX_LINE_LENGTH, f) == NULL)
00201             return MM_PREMATURE_EOF;
00202     }while (line[0] == '%');
00203
00204     /* line[] is either blank or has M,N, nz */
00205     if (sscanf(line, "%d %d %d", M, N, nz) == 3)
00206         return 0;
00207
00208     else
00209     do
00210     {
00211         num_items_read = fscanf(f, "%d %d %d", M, N, nz);
00212         if (num_items_read == EOF) return MM_PREMATURE_EOF;
00213     }
00214     while (num_items_read != 3);
00215
00216     return 0;
00217 }
00218
00219
00220 int mm_read_mtx_array_size(FILE *f, int *M, int *N)
00221 {
00222     char line[MM_MAX_LINE_LENGTH];
00223     int num_items_read;
00224     /* set return null parameter values, in case we exit with errors */
00225     *M = *N = 0;
00226
00227     /* now continue scanning until you reach the end-of-comments */
00228     do
00229     {
00230         if (fgets(line, MM_MAX_LINE_LENGTH, f) == NULL)
00231             return MM_PREMATURE_EOF;
00232     }while (line[0] == '%');
00233
00234     /* line[] is either blank or has M,N, nz */
00235     if (sscanf(line, "%d %d", M, N) == 2)
00236         return 0;
00237
00238     else /* we have a blank line */
00239     do
00240     {
00241         num_items_read = fscanf(f, "%d %d", M, N);
00242         if (num_items_read == EOF) return MM_PREMATURE_EOF;
00243     }
00244     while (num_items_read != 2);

```

```

00245
00246     return 0;
00247 }
00248
00249 int mm_write_mtx_array_size(FILE *f, int M, int N)
00250 {
00251     if (fprintf(f, "%d %d\n", M, N) != 2)
00252         return MM_COULD_NOT_WRITE_FILE;
00253     else
00254         return 0;
00255 }
00256
00257
00258
00259 /*-----*/
00260
00261 /*****
00262  * use when I[], J[], and val[]J, and val[] are already allocated */
00263 /*****
00264  */
00265 int mm_read_mtx_crd_data(FILE *f, int M, int N, int nz, int I[], int J[],
00266     double val[], MM_typecode matcode)
00267 {
00268     int i;
00269     if (mm_is_complex(matcode))
00270     {
00271         for (i=0; i<nz; i++)
00272             if (fscanf(f, "%d %d %lg %lg", &I[i], &J[i], &val[2*i], &val[2*i+1])
00273                 != 4) return MM_PREMATURE_EOF;
00274     }
00275     else if (mm_is_real(matcode))
00276     {
00277         for (i=0; i<nz; i++)
00278         {
00279             if (fscanf(f, "%d %d %lg\n", &I[i], &J[i], &val[i])
00280                 != 3) return MM_PREMATURE_EOF;
00281         }
00282     }
00283 }
00284
00285 else if (mm_is_pattern(matcode))
00286 {
00287     for (i=0; i<nz; i++)
00288         if (fscanf(f, "%d %d", &I[i], &J[i])
00289             != 2) return MM_PREMATURE_EOF;
00290 }
00291 else
00292     return MM_UNSUPPORTED_TYPE;
00293
00294 return 0;
00295 }
00296 }
00297
00298 int mm_read_mtx_crd_entry(FILE *f, int *I, int *J,
00299     double *real, double *imag, MM_typecode matcode)
00300 {
00301     if (mm_is_complex(matcode))
00302     {
00303         if (fscanf(f, "%d %d %lg %lg", I, J, real, imag)
00304             != 4) return MM_PREMATURE_EOF;
00305     }
00306     else if (mm_is_real(matcode))
00307     {
00308         if (fscanf(f, "%d %d %lg\n", I, J, real)
00309             != 3) return MM_PREMATURE_EOF;
00310     }
00311 }
00312
00313 else if (mm_is_pattern(matcode))
00314 {
00315     if (fscanf(f, "%d %d", I, J) != 2) return MM_PREMATURE_EOF;
00316 }
00317 else
00318     return MM_UNSUPPORTED_TYPE;
00319
00320 return 0;
00321 }
00322 }
00323
00324
00325 /*****
00326  mm_read_mtx_crd() fills M, N, nz, array of values, and return
00327  type code, e.g. 'MCRS'
00328
00329  if matrix is complex, values[] is of size 2*nz,
00330  (nz pairs of real/imaginary values)
00331  *****/

```

```

00332
00333 int mm_read_mtx_crd(char *fname, int *M, int *N, int *nz, int **I, int **J,
00334                     double **val, MM_typecode *matcode)
00335 {
00336     int ret_code;
00337     FILE *f;
00338
00339     if (strcmp(fname, "stdin") == 0) f=stdin;
00340     else
00341     if ((f = fopen(fname, "r")) == NULL)
00342         return MM_COULD_NOT_READ_FILE;
00343
00344
00345     if ((ret_code = mm_read_banner(f, matcode)) != 0)
00346         return ret_code;
00347
00348     if (!(mm_is_valid(*matcode) && mm_is_sparse(*matcode) &&
00349           mm_is_matrix(*matcode)))
00350         return MM_UNSUPPORTED_TYPE;
00351
00352     if ((ret_code = mm_read_mtx_crd_size(f, M, N, nz)) != 0)
00353         return ret_code;
00354
00355
00356     *I = (int *) malloc(*nz * sizeof(int));
00357     *J = (int *) malloc(*nz * sizeof(int));
00358     *val = NULL;
00359
00360     if (mm_is_complex(*matcode))
00361     {
00362         *val = (double *) malloc(*nz * 2 * sizeof(double));
00363         ret_code = mm_read_mtx_crd_data(f, *M, *N, *nz, *I, *J, *val,
00364                                         *matcode);
00365         if (ret_code != 0) return ret_code;
00366     }
00367     else if (mm_is_real(*matcode))
00368     {
00369         *val = (double *) malloc(*nz * sizeof(double));
00370         ret_code = mm_read_mtx_crd_data(f, *M, *N, *nz, *I, *J, *val,
00371                                         *matcode);
00372         if (ret_code != 0) return ret_code;
00373     }
00374
00375     else if (mm_is_pattern(*matcode))
00376     {
00377         ret_code = mm_read_mtx_crd_data(f, *M, *N, *nz, *I, *J, *val,
00378                                         *matcode);
00379         if (ret_code != 0) return ret_code;
00380     }
00381
00382     if (f != stdin) fclose(f);
00383     return 0;
00384 }
00385
00386 int mm_write_banner(FILE *f, MM_typecode matcode)
00387 {
00388     char *str = mm_typecode_to_str(matcode);
00389     int ret_code;
00390
00391     ret_code = fprintf(f, "%s %s\n", MatrixMarketBanner, str);
00392     free(str);
00393     if (ret_code != 2 )
00394         return MM_COULD_NOT_WRITE_FILE;
00395     else
00396         return 0;
00397 }
00398
00399 int mm_write_mtx_crd(char fname[], int M, int N, int nz, int I[], int J[],
00400                     double val[], MM_typecode matcode)
00401 {
00402     FILE *f;
00403     int i;
00404
00405     if (strcmp(fname, "stdout") == 0)
00406         f = stdout;
00407     else
00408     if ((f = fopen(fname, "w")) == NULL)
00409         return MM_COULD_NOT_WRITE_FILE;
00410
00411     /* print banner followed by typecode */
00412     fprintf(f, "%s ", MatrixMarketBanner);
00413     fprintf(f, "%s\n", mm_typecode_to_str(matcode));
00414
00415     /* print matrix sizes and nonzeros */
00416     fprintf(f, "%d %d %d\n", M, N, nz);
00417
00418     /* print values */

```

```

00419     if (mm_is_pattern(matcode))
00420         for (i=0; i<nz; i++)
00421             fprintf(f, "%d %d\n", I[i], J[i]);
00422     else
00423     if (mm_is_real(matcode))
00424         for (i=0; i<nz; i++)
00425             fprintf(f, "%d %d %20.16g\n", I[i], J[i], val[i]);
00426     else
00427     if (mm_is_complex(matcode))
00428         for (i=0; i<nz; i++)
00429             fprintf(f, "%d %d %20.16g %20.16g\n", I[i], J[i], val[2*i],
00430                     val[2*i+1]);
00431     else
00432     {
00433         if (f != stdout) fclose(f);
00434         return MM_UNSUPPORTED_TYPE;
00435     }
00436
00437     if (f !=stdout) fclose(f);
00438
00439     return 0;
00440 }
00441
00442 char *mm_strdup(const char *s)
00443 {
00444     int len = strlen(s);
00445     char *s2 = (char *) malloc((len+1)*sizeof(char));
00446     return strcpy(s2, s);
00447 }
00448
00449 char *mm_typecode_to_str(MM_typecode matcode)
00450 {
00451     char buffer[MM_MAX_LINE_LENGTH];
00452     char *types[4];
00453     char *mm_strdup(const char *);
00454     int error =0;
00455
00456     /* check for MTX type */
00457     if (mm_is_matrix(matcode))
00458         types[0] = MM_MTX_STR;
00459     else
00460         error=1;
00461
00462     /* check for CRD or ARR matrix */
00463     if (mm_is_sparse(matcode))
00464         types[1] = MM_SPARSE_STR;
00465     else
00466         if (mm_is_dense(matcode))
00467             types[1] = MM_DENSE_STR;
00468     else
00469         return NULL;
00470
00471     /* check for element data type */
00472     if (mm_is_real(matcode))
00473         types[2] = MM_REAL_STR;
00474     else
00475     if (mm_is_complex(matcode))
00476         types[2] = MM_COMPLEX_STR;
00477     else
00478     if (mm_is_pattern(matcode))
00479         types[2] = MM_PATTERN_STR;
00480     else
00481     if (mm_is_integer(matcode))
00482         types[2] = MM_INT_STR;
00483     else
00484         return NULL;
00485
00486     /* check for symmetry type */
00487     if (mm_is_general(matcode))
00488         types[3] = MM_GENERAL_STR;
00489     else
00490     if (mm_is_symmetric(matcode))
00491         types[3] = MM_SYMM_STR;
00492     else
00493     if (mm_is_hermitian(matcode))
00494         types[3] = MM_HERM_STR;
00495     else
00496     if (mm_is_skew(matcode))
00497         types[3] = MM_SKEW_STR;
00498     else
00499         return NULL;
00500
00501     sprintf(buffer,"%s %s %s %s", types[0], types[1], types[2], types[3]);
00502     return mm_strdup(buffer);
00503 }
00504
00505
00506
00507
00508
00509
00510

```

```
00511 }
```

5.35 print2D.cpp

```
00001 #include <iostream>
00002 #include <../inc/dense.hpp>
00003 using namespace dense;
00004
00005 template<typename T>
00006 void matrix<T>::print2D()
00007 {
00008     if (dim_size.size() == 2)
00009     {
00010         std::cout << "[" ;
00011         int idx = 0;
00012
00013         for (int i = 0; i < dim_size[0]; ++i)
00014         {
00015             for (int j = 0; j < dim_size[1]; ++j)
00016             {
00017                 std::cout << values[idx];
00018                 if (j != dim_size[1] - 1)
00019                 {
00020                     std::cout << ", ";
00021                 }
00022                 idx++;
00023             }
00024
00025             if (i != dim_size[0] - 1)
00026             {
00027                 std::cout << "; ";
00028             }
00029         }
00030         std::cout << "]\n" << std::endl;
00031     }
00032 }
00033
00034 template void dense::matrix<float>::print2D();
00035 template void dense::matrix<double>::print2D();
00036 template void dense::matrix<std::complex<float>>::print2D();
00037 template void dense::matrix<std::complex<double>>::print2D();
```

5.36 print_matrix.cpp

```
00001 #include " ../inc/sparse.hpp"
00002 using namespace sparse;
00003
00004 template <typename T>
00005 void matrix<T>::print_matrix()
00006 {
00007     for(int i=0; i<row_size; i++){
00008         for (int j=0; j<col_size;j++){
00009             std::cout<<(*this)(i,j)<<" ";
00010         }
00011         std::cout<<std::endl;
00012     }
00013     std::cout<<std::endl;
00014 }
00015
00016 template void sparse::matrix<float>::print_matrix();
00017 template void sparse::matrix<double>::print_matrix();
00018 template void sparse::matrix<std::complex<float>>::print_matrix();
00019 template void sparse::matrix<std::complex<double>>::print_matrix();
```

5.37 scal_mn.cpp

```
00001 #include "dense.hpp"
00002 using namespace dense;
00003
00004 template <typename T>
00005 matrix<T> matrix<T>:: operator*(T x){
00006     matrix<T> B = *this;
00007
00008     if constexpr (std::is_same_v<T, float>)
00009         sscal(B.values.size(), x, B.values.data(), 1);
```

```

00010     else if constexpr (std::is_same_v<T, double>)
00011         dscal(B.values.size(), x, B.values.data(), 1);
00012     else if constexpr (std::is_same_v<T, std::complex<float> >)
00013         cscal(B.values.size(), &x, B.values.data(), 1);
00014     else if constexpr (std::is_same_v<T, std::complex<double> >)
00015         zscal(B.values.size(), &x, B.values.data(), 1);
00016     return B;
00017 }
00018 template class dense::matrix<float>;
00019 template class dense::matrix<double>;
00020 template class dense::matrix<std::complex<float> >;
00021 template class dense::matrix<std::complex<double> >;

```

5.38 scal_nm.cpp

```

00001 #include "dense.hpp"
00002
00003 namespace dense {
00004
00005 template <typename T, typename U>
00006 matrix<T> operator*(U x, const matrix<T> & A) {
00007     dense::matrix<T> B = A;
00008
00009     if constexpr (std::is_same_v<T, float>)
00010         sscal(B.get_size(), x, B.get_values_data(), 1);
00011     else if constexpr (std::is_same_v<T, double>)
00012         dscal(B.get_size(), x, B.get_values_data(), 1);
00013     else if constexpr (std::is_same_v<T, std::complex<float> >)
00014         cscal(B.get_size(), &x, B.get_values_data(), 1);
00015     else if constexpr (std::is_same_v<T, std::complex<double> >)
00016         zscal(B.get_size(), &x, B.get_values_data(), 1);
00017     return B;
00018 }
00019 template dense::matrix<float> operator*(int x, const dense::matrix<float> &);
00020 template dense::matrix<float> operator*(float x, const dense::matrix<float> &);
00021 template dense::matrix<float> operator*(double x, const dense::matrix<float> &);
00022
00023 template dense::matrix<double> operator*(int x, const dense::matrix<double> &);
00024 template dense::matrix<double> operator*(float x, const dense::matrix<double> &);
00025 template dense::matrix<double> operator*(double x, const dense::matrix<double> &);
00026
00027 template dense::matrix<std::complex<float> > operator*(int x, const
dense::matrix<std::complex<float> > &);
00028 template dense::matrix<std::complex<float> > operator*(float x, const
dense::matrix<std::complex<float> > &);
00029 template dense::matrix<std::complex<float> > operator*(double x, const
dense::matrix<std::complex<float> > &);
00030 template dense::matrix<std::complex<float> > operator*(std::complex<float> x, const
dense::matrix<std::complex<float> > &);
00031 template dense::matrix<std::complex<float> > operator*(std::complex<double> x, const
dense::matrix<std::complex<float> > &);
00032
00033 template dense::matrix<std::complex<double> > operator*(int x, const
dense::matrix<std::complex<double> > &);
00034 template dense::matrix<std::complex<double> > operator*(float x, const
dense::matrix<std::complex<double> > &);
00035 template dense::matrix<std::complex<double> > operator*(double x, const
dense::matrix<std::complex<double> > &);
00036 template dense::matrix<std::complex<double> > operator*(std::complex<float> x, const
dense::matrix<std::complex<double> > &);
00037 template dense::matrix<std::complex<double> > operator*(std::complex<double> x, const
dense::matrix<std::complex<double> > &);}

```

5.39 set_value_at.cpp

```

00001 #include <../inc/dense.hpp>
00002 using namespace dense;
00003
00009 template <typename T>
00010 void dense::matrix<T>::set_value_at(size_t index, T value)
00011 {
00012     values.at(index) = value;
00013 }
00014
00015 template void dense::matrix<int>::set_value_at(size_t index, int value);
00016 template void dense::matrix<float>::set_value_at(size_t index, float value);
00017 template void dense::matrix<double>::set_value_at(size_t index, double value);
00018 template void dense::matrix<std::complex<float> >::set_value_at(size_t index, std::complex<float>
value);
00019 template void dense::matrix<std::complex<double> >::set_value_at(size_t index, std::complex<double>
value);

```

5.40 sparse-diagonal-getter.cpp

```

00001 #include "../inc/sparse.hpp"
00002 using namespace sparse;
00003
00004 template <typename T>
00005 std::vector<T> matrix<T>::diag() const {
00006     std::vector<T> ret;
00007     for (int i = 0; i < col_size; i++)
00008         ret.push_back(this->get_value_at(i * col_size + i));
00009     return ret;
00010 }
00011
00012 template class sparse::matrix<float>;
00013 template class sparse::matrix<double>;
00014 template class sparse::matrix<std::complex<float>>;
00015 template class sparse::matrix<std::complex<double>>;

```

5.41 sparse-get-value.cpp

```

00001 #include "../inc/sparse.hpp"
00002 using namespace sparse;
00003
00004 template <typename T>
00005 T matrix<T>::get_value_at(size_t index) const{
00006     //index = c + col_size*r
00007     size_t c = index%col_size;
00008     size_t r = index/col_size;
00009     size_t row_start = row_idx[r];
00010     size_t row_end = row_idx[r+1];
00011
00012     for (size_t i = row_start; i<row_end; i++){
00013         if (col_idx[i] == c){
00014             return values[i];
00015         }
00016     }
00017     return T{};
00018 }
00019 template class sparse::matrix<float>;
00020 template class sparse::matrix<double>;
00021 template class sparse::matrix<std::complex<float>>;
00022 template class sparse::matrix<std::complex<double>>;

```

5.42 sparse-parentheses-operator.cpp

```

00001 #include "../inc/sparse.hpp"
00002 using namespace sparse;
00003
00004 template <typename T>
00005 const T matrix<T>::operator () (size_t r, size_t c) const{
00006     size_t row_start = row_idx[r];
00007     size_t row_end = row_idx[r+1];
00008
00009     for (size_t i = row_start; i<row_end; i++){
00010         if (col_idx[i] == c){
00011             return values[i];
00012         }
00013     }
00014     return T{};
00015 }
00016 template class sparse::matrix<float>;
00017 template class sparse::matrix<double>;
00018 template class sparse::matrix<std::complex<float>>;
00019 template class sparse::matrix<std::complex<double>>;

```

5.43 sparse-set-value.cpp

```

00001 #include "../inc/sparse.hpp"
00002 using namespace sparse;
00003
00004 template <typename T>
00005 void matrix<T>::set_value_at(size_t index, T value){
00006     //index = c + col_size*r
00007     size_t c = index%col_size;
00008     size_t r = index/col_size;

```

```

00009     size_t row_start = row_idx[r];
00010     size_t row_end = row_idx[r+1];
00011
00012     for (size_t i = row_start; i<row_end; i++){
00013         if (col_idx[i] == c){
00014             values[i] = value;
00015             valid_handle = false;
00016             return;
00017         }
00018     }
00019
00020     throw std::invalid_argument("Can only set non-zero values");
00021 }
00022 template class sparse::matrix<float>;
00023 template class sparse::matrix<double>;
00024 template class sparse::matrix<std::complex<float>>;
00025 template class sparse::matrix<std::complex<double>>;

```

5.44 sparse_add.cpp

```

00001 #include "../inc/sparse.hpp"
00002 using namespace sparse;
00003
00004 template <typename T>
00005 sparse::matrix<T> sparse::matrix<T>::operator+(sparse::matrix<T> const& A){
00006     // refresh handles if required
00007     if (!valid_handle)
00008         this->refresh_handle();
00009     if (!A.valid_handle)
00010         A.refresh_handle();
00011     this->valid_handle = true;
00012     A.valid_handle = true;
00013     sparse_matrix_t ret;
00014     if constexpr (std::is_same_v<T, float>) {
00015         const float a = 1;
00016         sparse_s_add(SPARSE_OPERATION_NON_TRANSPOSE, A.handle, a, this->handle, &ret);
00017     }
00018     else if constexpr (std::is_same_v<T, double>) {
00019         const double a = 1;
00020         sparse_d_add(SPARSE_OPERATION_NON_TRANSPOSE, A.handle, a, this->handle, &ret);
00021     }
00022     else if constexpr (std::is_same_v<T, std::complex<float>>) {
00023         const MKL_Complex8 a = {1.0f, 0.0f};
00024         sparse_c_add(SPARSE_OPERATION_NON_TRANSPOSE, A.handle, a, this->handle, &ret);
00025     }
00026     else if constexpr (std::is_same_v<T, std::complex<double>>) {
00027         const MKL_Complex16 a = {1.0, 0.0};
00028         sparse_z_add(SPARSE_OPERATION_NON_TRANSPOSE, A.handle, a, this->handle, &ret);
00029     }
00030     return sparse::matrix<T>(ret, row_size, col_size, false);
00031 }
00032 }
00033 template class sparse::matrix<float>;
00034 template class sparse::matrix<double>;
00035 template class sparse::matrix<std::complex<float>>;
00036 template class sparse::matrix<std::complex<double>>;

```

5.45 sparse_assignment_operator.cpp

```

00001 #include "../inc/sparse.hpp"
00002 using namespace sparse;
00003
00004 template <typename T>
00005 matrix<T>& matrix<T>::operator=(matrix const& A)
00006 {
00007     if (this == &A) return *this; // self-assignment guard
00008
00009     values = A.values;
00010     col_idx = A.col_idx;
00011     row_idx = A.row_idx;
00012     row_size = A.row_size;
00013     col_size = A.col_size;
00014     ownership = A.ownership;
00015
00016     // Destroy existing handle if it was valid
00017     if ( valid_handle && handle != nullptr)
00018     {
00019         mkl_sparse_destroy(handle);
00020     }

```



```

00021
00022     handle = nullptr;
00023     valid_handle = false;
00024
00025     return *this;
00026 }
00027
00028 template class sparse::matrix<float>;
00029 template class sparse::matrix<double>;
00030 template class sparse::matrix<std::complex<float>>;
00031 template class sparse::matrix<std::complex<double>>;

```

5.46 sparse_banded_matrix.cpp

```

00001 #include "../inc/sparse.hpp"
00002 using namespace sparse;
00003
00004 template <typename T>
00005 matrix<T> matrix<T>::banded(int N, int kl, int ku, T val)
00006 {
00007     // Estimated memory (over-estimation) for reservation
00008     MKL_INT estimated_nnz = N * (ku + kl + 1);
00009
00010     std::vector<T> values;
00011     std::vector<MKL_INT> col_idx;
00012     std::vector<MKL_INT> row_idx(N+1);
00013
00014     // Reserve memory
00015     values.reserve(estimated_nnz);
00016     col_idx.reserve(estimated_nnz);
00017
00018     int current_nnz_count = 0;
00019
00020     for (int i = 0; i < N; i++)
00021     {
00022         row_idx[i] = current_nnz_count;
00023
00024         int start_col = std::max(0, i - kl);
00025         int end_col = std::min(N - 1, i + ku);
00026
00027         for (int j = start_col; j <= end_col; j++)
00028         {
00029             values.push_back(val);
00030             col_idx.push_back(j);
00031             current_nnz_count++;
00032         }
00033     }
00034
00035     row_idx[N] = current_nnz_count; // The last element of the row_idx vector in CSR must always be
nnz, // i.e. the size of the value vector
00036
00037     // Shrink the over-estimated reserved memory
00038     values.shrink_to_fit();
00039     col_idx.shrink_to_fit();
00040
00041     return matrix<T>(values, col_idx, row_idx, N, N);
00042 }
00043
00044 template class sparse::matrix<float>;
00045 template class sparse::matrix<double>;
00046 template class sparse::matrix<std::complex<float>>;
00047 template class sparse::matrix<std::complex<double>>;

```

5.47 sparse_comparison_operator.cpp

```

00001 #include "../inc/sparse.hpp"
00002 using namespace sparse;
00003
00004 template <typename T>
00005 bool matrix<T>::operator==(matrix const& A)
00006 {
00007     // Check for self-assignment
00008     if (this == &A) return true;
00009
00010     // Dimensions check
00011     if (row_size != A.row_size || col_size != A.col_size) return false;
00012
00013     // Check CSR vectors

```

```

00014     if (values != A.values) return false;
00015     if (row_idx != A.row_idx) return false;
00016     if (col_idx != A.col_idx) return false;
00017
00018     return true;
00019 }
00020
00021 template bool sparse::matrix<float>::operator==(matrix const& A);
00022 template bool sparse::matrix<double>::operator==(matrix const& A);
00023 template bool sparse::matrix<std::complex<float>>::operator==(matrix const& A);
00024 template bool sparse::matrix<std::complex<double>>::operator==(matrix const& A);

```

5.48 sparse_constant_mul.cpp

```

00001 #include "../inc/sparse.hpp"
00002 using namespace sparse;
00003
00004 template <typename T>
00005 sparse::matrix<T> sparse::matrix<T>::operator*(T x){
00006     const MKL_INT n = this->row_size*this->col_size;
00007     std::vector<T> n_vals(this->values); //_scal operates in place
00008     if constexpr (std::is_same_v<T, float>)
00009         sscal(n, x, n_vals.data(), 1);
00010     else if constexpr (std::is_same_v<T, double>)
00011         dscal(n, x, n_vals.data(), 1);
00012     else if constexpr (std::is_same_v<T, std::complex<float>>)
00013         cscal(n, &x, n_vals.data(), 1);
00014     else if constexpr (std::is_same_v<T, std::complex<double>>)
00015         zscal(n, &x, n_vals.data(), 1);
00016     //reused code from scal_mn.cpp
00017     return sparse::matrix<T>(n_vals, this->col_idx, this->row_idx, this->row_size, this->col_size);
00018 }
00019
00020 template class sparse::matrix<float>;
00021 template class sparse::matrix<double>;
00022 template class sparse::matrix<std::complex<float>>;
00023 template class sparse::matrix<std::complex<double>>;

```

5.49 sparse_copy_constructor.cpp

```

00001 #include "../inc/sparse.hpp"
00002 using namespace sparse;
00003
00004 template <typename T>
00005 matrix<T>::matrix (matrix const& A)
00006     :values(A.values), col_idx(A.col_idx), row_idx(A.row_idx),
00007     row_size(A.row_size), col_size(A.col_size), ownership(A.ownership)
00008 {
00009     handle = nullptr;
00010     valid_handle = false;
00011 }
00012 template class sparse::matrix<float>;
00013 template class sparse::matrix<double>;
00014 template class sparse::matrix<std::complex<float>>;
00015 template class sparse::matrix<std::complex<double>>;

```

5.50 sparse_handle_constructor.cpp

```

00001 #include "../inc/sparse.hpp"
00002 #include <algorithm>
00003 using namespace sparse;
00004
00005 template <typename T>
00006 matrix<T>::matrix(const sparse_matrix_t h, size_t rows, size_t cols, bool own){
00007     this->handle = h;
00008     this->valid_handle = true;
00009     this->ownership = own;
00010     this->row_size = rows;
00011     this->col_size = cols;
00012
00013
00014     sparse_index_base_t indexing;
00015     MKL_INT mkl_rows, mkl_cols;
00016     MKL_INT* row_start;
00017     MKL_INT* row_end;

```

```

00018     MKL_INT* col_idx_ptr;
00019
00020     if constexpr(std::is_same_v<T, float>){
00021         float* val_ptr;
00022         sparse_s_export_csr(h, &indexing, &mkl_rows, &mkl_cols,
00023             &row_start, &row_end, &col_idx_ptr, &val_ptr);
00024         this->values = std::vector<float>(val_ptr, val_ptr + (row_end[mkl_rows-1] - row_start[0]));
00025     }
00026     else if constexpr(std::is_same_v<T, double>){
00027         double* val_ptr;
00028         sparse_d_export_csr(h, &indexing, &mkl_rows, &mkl_cols,
00029             &row_start, &row_end, &col_idx_ptr, &val_ptr);
00030         this->values = std::vector<double>(val_ptr, val_ptr + (row_end[mkl_rows-1] - row_start[0]));
00031     }
00032     else if constexpr(std::is_same_v<T, std::complex<float>>){
00033         MKL_Complex8* val_ptr;
00034         sparse_c_export_csr(h, &indexing, &mkl_rows, &mkl_cols,
00035             &row_start, &row_end, &col_idx_ptr, &val_ptr);
00036         this->values = std::vector<std::complex<float>>(
00037             reinterpret_cast<std::complex<float>>*>(val_ptr),
00038             reinterpret_cast<std::complex<float>>*>(val_ptr + (row_end[mkl_rows-1] - row_start[0]))
00039         );
00040     }
00041     else if constexpr(std::is_same_v<T, std::complex<double>>){
00042         MKL_Complex16* val_ptr;
00043         sparse_z_export_csr(h, &indexing, &mkl_rows, &mkl_cols,
00044             &row_start, &row_end, &col_idx_ptr, &val_ptr);
00045         this->values = std::vector<std::complex<double>>(
00046             reinterpret_cast<std::complex<double>>*>(val_ptr),
00047             reinterpret_cast<std::complex<double>>*>(val_ptr + (row_end[mkl_rows-1] - row_start[0]))
00048         );
00049     }
00050
00051     this->col_idx = std::vector<MKL_INT>(col_idx_ptr,
00052         col_idx_ptr + (row_end[mkl_rows-1] - row_start[0]));
00053
00054     std::vector<MKL_INT> row_idx(mkl_rows+1);
00055     row_idx[0] = row_start[0];
00056     for(size_t i=0; i<mkl_rows; i++){
00057         row_idx[i+1] = row_end[i];
00058     }
00059     this->row_idx = std::move(row_idx);
00060 }
00061
00062 template class sparse::matrix<float>;
00063 template class sparse::matrix<double>;
00064 template class sparse::matrix<std::complex<float>>;
00065 template class sparse::matrix<std::complex<double>>;
00066

```

5.51 sparse_identity_matrix.cpp

```

00001 #include "../inc/sparse.hpp"
00002 using namespace sparse;
00003
00004 template <typename T>
00005 matrix<T> matrix<T>::I(int N)
00006 {
00007     std::vector<T> values(N, 1);
00008     std::vector<MKL_INT> col_idx(N);
00009     std::vector<MKL_INT> row_idx(N + 1);
00010
00011     for (int count = 0; count < N; count++)
00012     {
00013         col_idx[count] = count;
00014         row_idx[count] = count;
00015     }
00016
00017     row_idx[N] = values.size(); // The last element of the row_idx vector in CSR must always be nnz,
00018     i.e. the size of the value vector
00019     return matrix<T>(values, col_idx, row_idx, N, N);
00020 }
00021
00022 template class sparse::matrix<float>;
00023 template class sparse::matrix<double>;
00024 template class sparse::matrix<std::complex<float>>;
00025 template class sparse::matrix<std::complex<double>>;

```

5.52 sparse_mm_dxs.cpp

```

00001 #include "matrix.hpp"
00002 #include <mkl.h>
00003
00004 namespace dense {
00005
00006 template<typename T>
00007 dense::matrix<T> operator*(const dense::matrix<T>& A, const sparse::matrix<T>& B) {
00008     B.refresh_handle();
00009
00010     // xisting sparse × dense operator
00011     auto result = (B.transpose() * A.transpose()).transpose();
00012     return result;
00013 }
00014 template dense::matrix<float> operator*(const dense::matrix<float>&, const sparse::matrix<float>&);
00015 template dense::matrix<double> operator*(const dense::matrix<double>&, const sparse::matrix<double>&);
00016 template dense::matrix<std::complex<float>> operator*(const dense::matrix<std::complex<float>&, const
00017     sparse::matrix<std::complex<float>&);
00017 template dense::matrix<std::complex<double>> operator*(const dense::matrix<std::complex<double>&,
00018     const sparse::matrix<std::complex<double>&);
00018 }

```

5.53 sparse_mm_sxd.cpp

```

00001 #include "matrix.hpp"
00002 #include <mkl.h>
00003
00004 namespace sparse{
00005 template<typename T>
00006 dense::matrix<T> operator*(const sparse::matrix<T>& B, const dense::matrix<T>& A) {
00007     B.refresh_handle();
00008     dense::matrix<T> C(B.shape().first, A.get_dim_size(1));
00009
00010     // Matrix description for mkl_sparse_?_mm
00011     matrix_descr descr;
00012     descr.type = SPARSE_MATRIX_TYPE_GENERAL;
00013     descr.mode = SPARSE_FILL_MODE_FULL;
00014     descr.diag = SPARSE_DIAG_NON_UNIT;
00015
00016     if constexpr(std::is_same_v<T, float>){
00017         float a = 1;
00018         float b = 0;
00019         sparse_s_mm(SPARSE_OPERATION_NON_TRANSPOSE, a, B.get_handle(), descr, SPARSE_LAYOUT_ROW_MAJOR,
00020             A.get_const_values_data(),
00021             A.get_dim_size(1), A.get_dim_size(1), b, C.get_values_data(), C.get_dim_size(1));
00022     } else if constexpr(std::is_same_v<T, double>){
00023         double a = 1;
00024         double b = 0;
00025         sparse_d_mm(SPARSE_OPERATION_NON_TRANSPOSE, a, B.get_handle(), descr, SPARSE_LAYOUT_ROW_MAJOR,
00026             A.get_const_values_data(),
00027             A.get_dim_size(1), A.get_dim_size(1), b, C.get_values_data(), C.get_dim_size(1));
00028     } else if constexpr(std::is_same_v<T, std::complex<float>>){
00029         MKL_Complex8 a = {1,0}, b = {0,0};
00030         sparse_c_mm(SPARSE_OPERATION_NON_TRANSPOSE, a, B.get_handle(), descr, SPARSE_LAYOUT_ROW_MAJOR,
00031             reinterpret_cast<const MKL_Complex8*>(A.get_const_values_data()), A.get_dim_size(1),
00032             A.get_dim_size(1), b,
00033             reinterpret_cast<MKL_Complex8*>(C.get_values_data()), C.get_dim_size(1));
00034     } else if constexpr(std::is_same_v<T, std::complex<double>>){
00035         MKL_Complex16 a = {1,0}, b = {0,0};
00036         sparse_z_mm(SPARSE_OPERATION_NON_TRANSPOSE, a, B.get_handle(), descr, SPARSE_LAYOUT_ROW_MAJOR,
00037             reinterpret_cast<const MKL_Complex16*>(A.get_const_values_data()),
00038             A.get_dim_size(1), A.get_dim_size(1), b,
00039             reinterpret_cast<MKL_Complex16*>(C.get_values_data()), C.get_dim_size(1));
00040     }
00041     return C;
00042 }
00043
00044 template dense::matrix<float> operator*(const sparse::matrix<float>&, const dense::matrix<float>&);
00045 template dense::matrix<double> operator*(const sparse::matrix<double>&, const dense::matrix<double>&);
00046 template dense::matrix<std::complex<float>> operator*(
00047     const sparse::matrix<std::complex<float>&, const dense::matrix<std::complex<float>&);
00048 template dense::matrix<std::complex<double>> operator*(
00049     const sparse::matrix<std::complex<double>&, const dense::matrix<std::complex<double>&);
00050 }

```

5.54 sparse_mul_nxs.cpp

```

00001 #include "sparse.hpp"

```

```

00002
00003 namespace sparse {
00004
00005 template <typename T, typename U>
00006 matrix<T> operator*(U x, const matrix<T>& A) {
00007     A.refresh_handle();
00008     matrix<T> result(A);
00009
00010     auto& vals = result.get_values_data();
00011
00012     #pragma omp parallel for simd schedule(static)
00013     for (size_t i = 0; i < vals.size(); ++i)
00014         vals[i] *= x;
00015     result.invalidate_handle();
00016     return result;
00017 }
00018 template sparse::matrix<float> operator*(int x, const sparse::matrix<float>&);
00019 template sparse::matrix<float> operator*(float x, const sparse::matrix<float>&);
00020 template sparse::matrix<float> operator*(double x, const sparse::matrix<float>&);
00021
00022 template sparse::matrix<double> operator*(int x, const sparse::matrix<double>&);
00023 template sparse::matrix<double> operator*(float x, const sparse::matrix<double>&);
00024 template sparse::matrix<double> operator*(double x, const sparse::matrix<double>&);
00025
00026 template sparse::matrix<std::complex<float>> operator*(int x, const
sparse::matrix<std::complex<float>>&);
00027 template sparse::matrix<std::complex<float>> operator*(float x, const
sparse::matrix<std::complex<float>>&);
00028 template sparse::matrix<std::complex<float>> operator*(double x, const
sparse::matrix<std::complex<float>>&);
00029 template sparse::matrix<std::complex<float>> operator*(std::complex<float> x, const
sparse::matrix<std::complex<float>>&);
00030 template sparse::matrix<std::complex<float>> operator*(std::complex<double> x, const
sparse::matrix<std::complex<float>>&);
00031
00032 template sparse::matrix<std::complex<double>> operator*(int x, const
sparse::matrix<std::complex<double>>&);
00033 template sparse::matrix<std::complex<double>> operator*(float x, const
sparse::matrix<std::complex<double>>&);
00034 template sparse::matrix<std::complex<double>> operator*(double x, const
sparse::matrix<std::complex<double>>&);
00035 template sparse::matrix<std::complex<double>> operator*(std::complex<float> x, const
sparse::matrix<std::complex<double>>&);
00036 template sparse::matrix<std::complex<double>> operator*(std::complex<double> x, const
sparse::matrix<std::complex<double>>&);

```

5.55 sparse_mul_sxn.cpp

```

00001 #include "../inc/sparse.hpp"
00002 using namespace sparse;
00003
00004 template <typename T>
00005 matrix<T> matrix<T>::operator*(T x) {
00006     if (!valid_handle)
00007         this->refresh_handle();
00008
00009     matrix<T> result(*this);
00010
00011     // Scale the nonzero values (OpenMP-parallelized & vectorized)
00012     #pragma omp parallel for simd schedule(static)
00013     for (size_t i = 0; i < result.values.size(); ++i)
00014         result.values[i] *= x;
00015     result.valid_handle = false;
00016     result.handle = nullptr;
00017
00018     return result;
00019 }
00020
00021 template class sparse::matrix<float>;
00022 template class sparse::matrix<double>;
00023 template class sparse::matrix<std::complex<float>>;
00024 template class sparse::matrix<std::complex<double>>;

```

5.56 sparse_mv.cpp

```

00001 #include "../inc/sparse.hpp"
00002 using namespace sparse;
00003
00004 template <typename T>

```

```

00005 std::vector<T> matrix<T>::operator*(const std::vector<T> x)
00006 {
00007     // Result vector
00008     std::vector<T> result(((*this).shape().first);
00009
00010     // Refresh or regenerate handle
00011     (*this).refresh_handle();
00012
00013     // Matrix description for mkl_sparse_mv
00014     matrix_descr descr;
00015     descr.type = SPARSE_MATRIX_TYPE_GENERAL;
00016     descr.mode = SPARSE_FILL_MODE_FULL;
00017     descr.diag = SPARSE_DIAG_NON_UNIT;
00018
00019     if constexpr(std::is_same_v<T, float>)
00020     {
00021         float a = 1.0;
00022         float b = 0;
00023         sparse_s_mv(SPARSE_OPERATION_NON_TRANSPOSE, a, (*this).handle, descr, x.data(), b,
00024 result.data());
00025         return result;
00026     }
00027     else if constexpr(std::is_same_v<T, double>)
00028     {
00029         double a = 1.0;
00030         double b = 0;
00031         sparse_d_mv(SPARSE_OPERATION_NON_TRANSPOSE, a, (*this).handle, descr, x.data(), b,
00032 result.data());
00033         return result;
00034     }
00035     else if constexpr(std::is_same_v<T, std::complex<float> >)
00036     {
00037         MKL_Complex8 a = {1.0f, 0.0f};
00038         MKL_Complex8 b = {0.0f, 0.0f};
00039         sparse_c_mv(SPARSE_OPERATION_NON_TRANSPOSE,
00040 a,
00041 (*this).handle,
00042 descr,
00043 reinterpret_cast<const MKL_Complex8*>(x.data()),
00044 b,
00045 reinterpret_cast<MKL_Complex8*>(result.data()));
00046
00047         return result;
00048     }
00049     else if constexpr(std::is_same_v<T, std::complex<double> >)
00050     {
00051         MKL_Complex16 a = {1.0, 0.0};
00052         MKL_Complex16 b = {0.0, 0.0};
00053         sparse_z_mv(SPARSE_OPERATION_NON_TRANSPOSE,
00054 a,
00055 (*this).handle,
00056 descr,
00057 reinterpret_cast<const MKL_Complex16*>(x.data()),
00058 b,
00059 reinterpret_cast<MKL_Complex16*>(result.data()));
00060
00061         return result;
00062     }
00063 }
00064
00065 template std::vector<float> matrix<float>::operator*(const std::vector<float> x);
00066 template std::vector<double> matrix<double>::operator*(const std::vector<double> x);
00067 template std::vector<std::complex<float> > matrix<std::complex<float> >::operator*(const
std::vector<std::complex<float> > x);
00068 template std::vector<std::complex<double> > matrix<std::complex<double> >::operator*(const
std::vector<std::complex<double> > x);

```

5.57 sparse_print2D.cpp

```

00001 #include "../inc/sparse.hpp"
00002 using namespace sparse;
00003
00004 template <typename T>
00005 void matrix<T>::print2D()
00006 {
00007     std::cout << "Values: " ;
00008     for (size_t n = 0; n < values.size(); n++)
00009     {
00010         std::cout << values[n] << " ";
00011     }
00012 }

```

```

00013     std::cout << "\nCol_idx: " ;
00014     for (size_t n = 0; n < col_idx.size(); n++)
00015     {
00016         std::cout << col_idx[n] << " ";
00017     }
00018
00019     std::cout << "\nRow_idx: " ;
00020     for (size_t n = 0; n < row_idx.size(); n++)
00021     {
00022         std::cout << row_idx[n] << " ";
00023     }
00024
00025     std::cout << std::endl;
00026 }
00027
00028 template void sparse::matrix<float>::print2D();
00029 template void sparse::matrix<double>::print2D();
00030 template void sparse::matrix<std::complex<float>>::print2D();
00031 template void sparse::matrix<std::complex<double>>::print2D();

```

5.58 sparse_readMM.cpp

```

00001 #include "../inc/sparse.hpp"
00002
00003 using namespace sparse;
00004
00005 template <typename T>
00006 matrix<T> matrix<T>::sparse_readMM(const std::string& MMpath)
00007 {
00008     FILE *f;
00009     MM_typecode matcode;
00010     int MMrows, MMcols, MMnz;
00011
00012     // Open the file
00013     if ((f=fopen(MMpath.c_str(), "r")) == NULL)
00014     {
00015         fclose(f);
00016         throw std::runtime_error("Error: Could not open file " + MMpath);
00017     }
00018
00019     // Read Banner
00020     if (mm_read_banner(f, &matcode) != 0)
00021     {
00022         fclose(f);
00023         throw std::runtime_error("Error: Could not process Matrix Market banner.");
00024     }
00025
00026     // Check if it's a valid sparse matrix
00027     if (!mm_is_matrix(matcode) || !mm_is_coordinate(matcode))
00028     {
00029         fclose(f);
00030         throw std::runtime_error("Error: This code only supports sparse coordinate matrices.");
00031     }
00032
00033     // Read Size (Rows, Cols, Non-zeros)
00034     if (mm_read_mtx_crd_size(f, &MMrows, &MMcols, &MMnz) != 0)
00035     {
00036         fclose(f);
00037         throw std::runtime_error("Error: Could not read matrix dimensions.");
00038     }
00039
00040     std::cout << "Successfully opened " << MMpath << std::endl;
00041     std::cout << "Rows: " << MMrows << ", Cols: " << MMcols << ", Non-Zeros: " << MMnz << std::endl;
00042
00043     // Allocate Memory Vectors
00044     std::vector<int> rows(static_cast<size_t>(MMnz));
00045     std::vector<int> cols(static_cast<size_t>(MMnz));
00046     std::vector<T> val(MMnz);
00047
00048     // Temp COO handle
00049     sparse_matrix_t coo_handle = nullptr;
00050
00051     if constexpr (std::is_same_v<T, float>)
00052     {
00053         for (int i = 0; i < MMnz; i++)
00054         {
00055             if (fscanf(f, "%d %d %f\n", &rows[i], &cols[i], &val[i]) != 3)
00056             {
00057                 fclose(f);
00058                 throw std::runtime_error("Error: File ended unexpectedly at line " +
00059                     std::to_string(i));
00060             }
00061             // Convert 1-based to 0-based for C

```

```

00061         rows[i]--;
00062         cols[i]--;
00063     }
00064     fclose(f);
00065
00066     mkl_sparse_s_create_coo(&coo_handle, SPARSE_INDEX_BASE_ZERO, MMrows, MMcols, MMnz,
rows.data(), cols.data(), val.data());
00067 }
00068 else if constexpr (std::is_same_v<T,double>)
00069 {
00070     // Read Data Loop
00071     for (int i = 0; i < MMnz; i++)
00072     {
00073         if(fscanf(f, "%d %d %lg\n", &rows[i], &cols[i], &val[i]) != 3)
00074         {
00075             throw std::runtime_error("Error: File ended unexpectedly at line " +
std::to_string(i));
00076         }
00077         // Convert 1-based to 0-based for C
00078         rows[i]--;
00079         cols[i]--;
00080     }
00081     fclose(f);
00082
00083     mkl_sparse_d_create_coo(&coo_handle, SPARSE_INDEX_BASE_ZERO, MMrows, MMcols, MMnz,
rows.data(), cols.data(), val.data());
00084 }
00085 else if constexpr (std::is_same_v<T,std::complex<float>)>
00086 {
00087     float realVal, imVal;
00088     for (int i = 0; i < MMnz; i++)
00089     {
00090         if(fscanf(f, "%d %d %f %f\n", &rows[i], &cols[i], &realVal, &imVal) != 4)
00091         {
00092             throw std::runtime_error("Error: File ended unexpectedly at line " +
std::to_string(i));
00093         }
00094         // Convert 1-based to 0-based for C
00095         rows[i]--;
00096         cols[i]--;
00097         val[i] = std::complex<float>(realVal, imVal);
00098     }
00099     fclose(f);
00100
00101     mkl_sparse_c_create_coo(&coo_handle, SPARSE_INDEX_BASE_ZERO, MMrows, MMcols, MMnz,
rows.data(), cols.data(), reinterpret_cast<MKL_Complex8*>(val.data()));
00102 }
00103 else if constexpr (std::is_same_v<T,std::complex<double>)>
00104 {
00105     double realVal, imVal;
00106     for (int i = 0; i < MMnz; i++)
00107     {
00108         if(fscanf(f, "%d %d %lg %lg\n", &rows[i], &cols[i], &realVal, &imVal) != 4)
00109         {
00110             throw std::runtime_error("Error: File ended unexpectedly at line " +
std::to_string(i));
00111         }
00112         // Convert 1-based to 0-based for C
00113         rows[i]--;
00114         cols[i]--;
00115         val[i] = std::complex<double>(realVal, imVal);
00116     }
00117     fclose(f);
00118
00119     mkl_sparse_z_create_coo(&coo_handle, SPARSE_INDEX_BASE_ZERO, MMrows, MMcols, MMnz,
rows.data(), cols.data(), reinterpret_cast<MKL_Complex16*>(val.data()));
00120 }
00121
00122     sparse_matrix_t csr_handle = nullptr;
00123     mkl_sparse_convert_csr(coo_handle, SPARSE_OPERATION_NON_TRANSPOSE, &csr_handle);
00124
00125     matrix<T> A = {csr_handle, static_cast<size_t>(MMrows), static_cast<size_t>(MMcols), true};
00126
00127     mkl_sparse_destroy(coo_handle); // Destroy temp COO handle
00128
00129     return A;
00130 }
00131
00132 template class sparse::matrix<float>;
00133 template class sparse::matrix<double>;
00134 template class sparse::matrix<std::complex<float>>;
00135 template class sparse::matrix<std::complex<double>>;

```


5.59 sparse_refresh_handle.cpp

```

00001 #include "../inc/sparse.hpp"
00002 using namespace sparse;
00003
00004 template <typename T>
00005 void matrix<T>::refresh_handle() const{
00006     if (this->valid_handle && this->handle != nullptr){ return; }
00007     if (this->handle != nullptr){ sparse_destroy(this->handle); }
00008
00009     sparse_status_t status = SPARSE_STATUS_SUCCESS;
00010
00011     MKL_INT* row_start = const_cast<MKL_INT*>(this->row_idx.data());
00012     MKL_INT* row_end   = const_cast<MKL_INT*>(this->row_idx.data() + 1);
00013     MKL_INT* col_indx  = const_cast<MKL_INT*>(this->col_idx.data());
00014
00015     if constexpr(std::is_same_v<T,float>){
00016         status = sparse_s_create_csr(&this->handle, SPARSE_INDEX_BASE_ZERO,
00017             (MKL_INT)this->row_size, (MKL_INT)this->col_size,
00018             row_start,row_end,col_indx,
00019             const_cast<float*>(this->values.data()));
00020     }
00021     else if constexpr(std::is_same_v<T,double>){
00022         status = sparse_d_create_csr(&this->handle, SPARSE_INDEX_BASE_ZERO,
00023             (MKL_INT)this->row_size, (MKL_INT)this->col_size,
00024             row_start,row_end,col_indx,
00025             const_cast<double*>(this->values.data()));
00026     }
00027     else if constexpr(std::is_same_v<T,std::complex<float>>){
00028         status = sparse_c_create_csr(&this->handle, SPARSE_INDEX_BASE_ZERO,
00029             (MKL_INT)this->row_size, (MKL_INT)this->col_size,
00030             row_start,row_end,col_indx,
00031             reinterpret_cast<MKL_Complex8*>(const_cast<std::complex<float>>*(this->values.data())));
00032     }
00033     else if constexpr(std::is_same_v<T,std::complex<double>>){
00034         status = sparse_z_create_csr(&this->handle, SPARSE_INDEX_BASE_ZERO,
00035             (MKL_INT)this->row_size, (MKL_INT)this->col_size,
00036             row_start,row_end,col_indx,
00037             reinterpret_cast<MKL_Complex16*>(const_cast<std::complex<double>>*(this->values.data())));
00038     }
00039
00040     this->valid_handle = (status == SPARSE_STATUS_SUCCESS);
00041     return;
00042 }
00043
00044 template class sparse::matrix<float>;
00045 template class sparse::matrix<double>;
00046 template class sparse::matrix<std::complex<float>>;
00047 template class sparse::matrix<std::complex<double>>;

```

5.60 sparse_shape.cpp

```

00001 #include "../inc/sparse.hpp"
00002 using namespace sparse;
00003
00004 template <typename T>
00005 std::pair<size_t, size_t> matrix<T>::shape() const{
00006     return {row_size, col_size};
00007 }
00008 template class sparse::matrix<float>;
00009 template class sparse::matrix<double>;
00010 template class sparse::matrix<std::complex<float>>;
00011 template class sparse::matrix<std::complex<double>>;

```

5.61 sparse_spmv.cpp

```

00001 #include "../inc/sparse.hpp"
00002 using namespace sparse;
00003
00004 template <typename T>
00005 matrix<T> matrix<T>::operator*(matrix<T> const& A){
00006     // refresh handles if required
00007     if (!valid_handle)
00008         this->refresh_handle();
00009     if (!A.valid_handle)
00010         A.refresh_handle();
00011     this->valid_handle = true;
00012     A.valid_handle = true;
00013     sparse_matrix_t ret;

```

```

00014     sparse_spmv(SPARSE_OPERATION_NON_TRANSPOSE, this->handle, A.handle, &ret);
00015     return matrix<T>(ret, this->row_size, A.col_size, false);
00016 }
00017 template class sparse::matrix<float>;
00018 template class sparse::matrix<double>;
00019 template class sparse::matrix<std::complex<float>>;
00020 template class sparse::matrix<std::complex<double>>;

```

5.62 sparse_sub.cpp

```

00001 #include "../inc/sparse.hpp"
00002 using namespace sparse;
00003
00004 template <typename T>
00005 sparse::matrix<T> sparse::matrix<T>::operator-(sparse::matrix<T> const& A){
00006     // refresh handles if required
00007     if (!valid_handle)
00008         this->refresh_handle();
00009     if (!A.valid_handle)
00010         A.refresh_handle();
00011     this->valid_handle = true;
00012     A.valid_handle = true;
00013     sparse_matrix_t ret;
00014     if constexpr (std::is_same_v<T, float>) {
00015         const float a = -1.0f;
00016         sparse_s_add(SPARSE_OPERATION_NON_TRANSPOSE, A.handle, a, this->handle, &ret);
00017     }
00018     else if constexpr (std::is_same_v<T, double>) {
00019         const double a = -1.0;
00020         sparse_d_add(SPARSE_OPERATION_NON_TRANSPOSE, A.handle, a, this->handle, &ret);
00021     }
00022     else if constexpr (std::is_same_v<T, std::complex<float>>) {
00023         const MKL_Complex8 a = {-1.0f, 0.0f};
00024         sparse_c_add(SPARSE_OPERATION_NON_TRANSPOSE, A.handle, a, this->handle, &ret);
00025     }
00026     else if constexpr (std::is_same_v<T, std::complex<double>>) {
00027         const MKL_Complex16 a = {-1.0, 0.0};
00028         sparse_z_add(SPARSE_OPERATION_NON_TRANSPOSE, A.handle, a, this->handle, &ret);
00029     }
00030     return sparse::matrix<T>(ret, row_size, col_size, false);
00031 }
00032 }
00033 template class sparse::matrix<float>;
00034 template class sparse::matrix<double>;
00035 template class sparse::matrix<std::complex<float>>;
00036 template class sparse::matrix<std::complex<double>>;

```

5.63 sparse_transpose.cpp

```

00001 #include "sparse.hpp"
00002 using namespace sparse;
00003
00004 template <typename T>
00005 matrix<T> matrix<T>::transpose() const{
00006     if (!valid_handle) refresh_handle();
00007
00008     sparse_matrix_t At_handle = nullptr;
00009     sparse_convert_csr(handle, SPARSE_OPERATION_TRANSPOSE, &At_handle);
00010
00011     sparse_index_base_t indexing;
00012     MKL_INT rows, cols;
00013     MKL_INT *row_start = nullptr, *row_end = nullptr, *col_ind = nullptr;
00014     T *vals = nullptr;
00015
00016     if constexpr (std::is_same_v<T, float>)
00017         sparse_s_export_csr(At_handle, &indexing, &rows, &cols, &row_start, &row_end, &col_ind,
00018 &vals);
00019     else if constexpr (std::is_same_v<T, double>)
00020         sparse_d_export_csr(At_handle, &indexing, &rows, &cols, &row_start, &row_end, &col_ind,
00021 &vals);
00022     else if constexpr (std::is_same_v<T, std::complex<float>>)
00023         sparse_c_export_csr(At_handle, &indexing, &rows, &cols, &row_start, &row_end, &col_ind,
00024 reinterpret_cast<MKL_Complex8 *>(&vals));
00025     else if constexpr (std::is_same_v<T, std::complex<double>>)
00026         sparse_z_export_csr(At_handle, &indexing, &rows, &cols, &row_start, &row_end, &col_ind,
00027 reinterpret_cast<MKL_Complex16 *>(&vals));
00028
00029     return matrix<T>(At_handle, rows, cols, true);
00030 }

```

```

00027
00028 template class sparse::matrix<float>;
00029 template class sparse::matrix<double>;
00030 template class sparse::matrix<std::complex<float>>;
00031 template class sparse::matrix<std::complex<double>>;

```

5.64 sparse_vxs.cpp

```

00001 #include "sparse.hpp"
00002
00003 namespace sparse {
00004
00005 template <typename T>
00006 std::vector<T> operator*(std::vector<T> x, const matrix<T>& A) {
00007     A.refresh_handle();
00008     std::vector<T> result(A.shape().second);
00009
00010     // Matrix description for mkl_sparse_?_mv
00011     matrix_descr descr;
00012     descr.type = SPARSE_MATRIX_TYPE_GENERAL;
00013     descr.mode = SPARSE_FILL_MODE_FULL;
00014     descr.diag = SPARSE_DIAG_NON_UNIT;
00015
00016     if constexpr(std::is_same_v<T, float>){
00017         sparse_s_mv(SPARSE_OPERATION_TRANSPOSE, 1.0f, A.get_handle(), descr, x.data(), 0.0f,
00018 result.data());
00019     }
00020     else if constexpr(std::is_same_v<T, double>){
00021         sparse_d_mv(SPARSE_OPERATION_TRANSPOSE, 1.0, A.get_handle(), descr, x.data(), 0.0,
00022 result.data());
00023     }
00024     else if constexpr(std::is_same_v<T, std::complex<float>>){
00025         MKL_Complex8 a = {1.0f, 0.0f};
00026         MKL_Complex8 b = {0.0f, 0.0f};
00027         sparse_c_mv(SPARSE_OPERATION_TRANSPOSE, a, A.get_handle(), descr, reinterpret_cast<const
00028 MKL_Complex8*>(x.data()), b, reinterpret_cast<MKL_Complex8*>(result.data()));
00029     }
00030     else if constexpr(std::is_same_v<T, std::complex<double>>){
00031         MKL_Complex16 a = {1.0, 0.0};
00032         MKL_Complex16 b = {0.0, 0.0};
00033         sparse_z_mv(SPARSE_OPERATION_TRANSPOSE, a, A.get_handle(), descr, reinterpret_cast<const
00034 MKL_Complex16*>(x.data()), b, reinterpret_cast<MKL_Complex16*>(result.data()));
00035     }
00036     return result;
00037 }
00038
00039 template std::vector<float> operator*<float>(std::vector<float>, const matrix<float>&);
00040 template std::vector<double> operator*<double>(std::vector<double>, const matrix<double>&);
00041 template std::vector<std::complex<float>> operator*<std::complex<float>>(std::vector<std::complex<float>>, const matrix<std::complex<float>>&);
00042 template std::vector<std::complex<double>> operator*<std::complex<double>>(std::vector<std::complex<double>>, const matrix<std::complex<double>>&);

```

5.65 svd.cpp

```

00001 #include "../inc/dense.hpp"
00002 #include <cstring>
00003 #include <variant>
00004 using namespace dense;
00005
00006 template <typename T>
00007 void matrix<T>::svd(matrix<T>& u, matrix<T>& s, matrix<T>& v) {
00008     MKL_INT info;
00009     MKL_INT n = dim_size[1];
00010     MKL_INT m = dim_size[0];
00011     MKL_INT lda = n;
00012     MKL_INT ldu = m;
00013     MKL_INT ldv = n;
00014
00015     size_t meml = n*m*sizeof(T);
00016     T* A_data = (T*) mkl_malloc(meml, 64);
00017     memcpy(A_data, values.data(), meml);
00018
00019     std::vector<T> u_vec(m*m, 0);
00020     std::vector<T> v_vec(n*n, 0);
00021     if constexpr(std::is_same_v<T, float>){
00022         std::vector<float> s_vec(n, 0);
00023         info = LAPACKE_sgesdd(LAPACK_ROW_MAJOR, 'A', m, n, A_data, lda, s_vec.data(),
00024 u_vec.data(), ldu, v_vec.data(), ldv);
00025         s = matrix<float>(s_vec, m, n);
00026     }

```

```

00025     }
00026     else if constexpr (std::is_same_v<T, double>){
00027         std::vector<double> s_vec (n,0);
00028         info = LAPACKE_dgesdd (LAPACK_ROW_MAJOR , 'A' , m, n ,A_data, lda, s_vec.data(), u_vec.data(),
ldu, v_vec.data(), ldv);
00029         s = matrix<double>(s_vec, m, n);
00030     }
00031     else if constexpr (std::is_same_v<T, std::complex<float> >){
00032         std::vector<float> s_vec (n,0);
00033         info = LAPACKE_cgesdd (LAPACK_ROW_MAJOR , 'A' , m, n , reinterpret_cast<MKL_Complex8*>
(A_data) , lda, s_vec.data(), reinterpret_cast<MKL_Complex8*> (u_vec.data()), ldu,
reinterpret_cast<MKL_Complex8*> (v_vec.data()), ldv);
00034         std::vector<std::complex<float> > s_vec_c;
00035         for(auto i: s_vec){
00036             s_vec_c.push_back(std::complex<float>(i,0));
00037         }
00038         s = matrix<std::complex<float> >(s_vec_c, m, n);
00039     }
00040     else if constexpr (std::is_same_v<T, std::complex<double> >){
00041         std::vector<double> s_vec (n,0);
00042         info = LAPACKE_zgesdd (LAPACK_ROW_MAJOR , 'A' , m, n , reinterpret_cast<MKL_Complex16*>
(A_data) , lda, s_vec.data(), reinterpret_cast<MKL_Complex16*> (u_vec.data()), ldu,
reinterpret_cast<MKL_Complex16*> (v_vec.data()), ldv);
00043         std::vector<std::complex<double> > s_vec_z;
00044         for(auto i: s_vec){
00045             s_vec_z.push_back(std::complex<double>(i,0));
00046         }
00047         s = matrix<std::complex<double> >(s_vec_z, m, n);
00048     }
00049     u = matrix<T>(u_vec, std::vector<size_t>{(size_t)(m), (size_t)(m)}, std::vector<int>(m, 1));
00050     v = matrix<T>(v_vec, std::vector<size_t>{(size_t)(n), (size_t)(n)}, std::vector<int>(n, 1));
00051 }
00052
00053 template class dense::matrix<float>;
00054 template class dense::matrix<double>;
00055 template class dense::matrix<std::complex<float> >;
00056 template class dense::matrix<std::complex<double> >;

```

5.66 transpose.cpp

```

00001 #include "dense.hpp"
00002 using namespace dense;
00003
00004 template <typename T>
00005 matrix<T> matrix<T>:: transpose() const{
00006
00007     // Transposed dimensions
00008     size_t rows = this->dim_size[1];
00009     size_t cols = this->dim_size[0];
00010
00011     matrix<T> At(rows, cols);
00012
00013     if constexpr (std::is_same_v<T, float>){
00014         float a = 1;
00015         somatcopy('R', 'T', rows, cols, a, this->values.data(), cols, At.values.data(), rows);}
00016     else if constexpr (std::is_same_v<T, double>){
00017         double a = 1;
00018         domatcopy('R', 'T', rows, cols, a, this->values.data(), cols, At.values.data(), rows);}
00019     else if constexpr (std::is_same_v<T, std::complex<float> >){
00020         MKL_Complex8 a = {1.0f, 0.0f};
00021         comatcopy('R', 'T', rows, cols, a,
reinterpret_cast<MKL_Complex8*>(const_cast<std::complex<float>*>(this->values.data())), cols,
reinterpret_cast<MKL_Complex8*>(At.values.data()), rows);}
00022     else if constexpr (std::is_same_v<T, std::complex<double> >){
00023         MKL_Complex16 a = {1.0, 0.0};
00024         zomatcopy('R', 'T', rows, cols, a,
reinterpret_cast<MKL_Complex16*>(const_cast<std::complex<double>*>(this->values.data())), cols,
reinterpret_cast<MKL_Complex16*>(At.values.data()), rows);}
00025     return At;
00026 }
00027 template class dense::matrix<float>;
00028 template class dense::matrix<double>;
00029 template class dense::matrix<std::complex<float> >;
00030 template class dense::matrix<std::complex<double> >;

```

5.67 vectors_constructor.cpp

```

00001 #include <../inc/dense.hpp>
00002 using namespace dense;

```

```
00003
00004 template<typename T>
00005 matrix<T>::matrix(std::vector<T> v, std::vector<size_t> d, std::vector<int> p)
00006     : values(v), dim_size(d), precomputed_strides(p)
00007 {
00008
00009 }
00010
00011 template class dense::matrix<float>;
00012 template class dense::matrix<double>;
00013 template class dense::matrix<std::complex<float>>>;
00014 template class dense::matrix<std::complex<double>>>;
```

